

SUPERNOVA: Design Strategies and Opportunities for Interactive Visualization in Computational Notebooks

Zijie J. Wang
jayw@gatech.edu
Georgia Tech
Atlanta, Georgia, USA

David Munechika
david.munechika@gatech.edu
Georgia Tech
Atlanta, Georgia, USA

Seongmin Lee
seongmin@gatech.edu
Georgia Tech
Atlanta, Georgia, USA

Duen Horng Chau
polo@gatech.edu
Georgia Tech
Atlanta, Georgia, USA

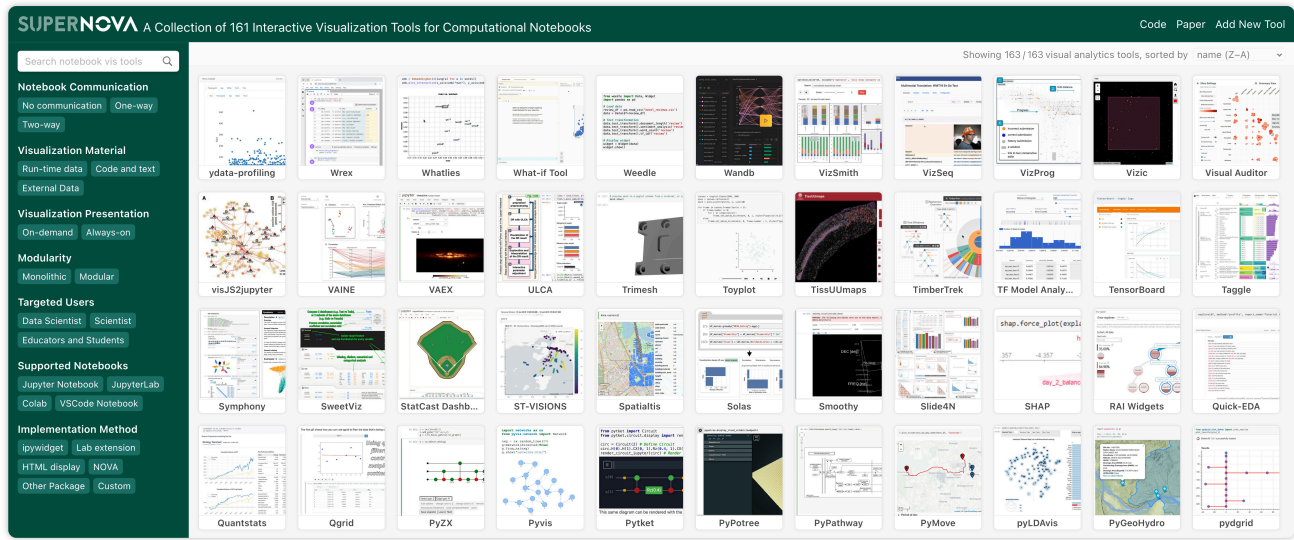


Fig. 1: SUPERNOVA is a browser for exploring 163 notebook interactive visualization tools. Users can filter and search for tools with specific properties in the left panel. Clicking on a tool reveals details including paper metadata and GitHub repository.

ABSTRACT

Computational notebooks, such as Jupyter Notebook, have become data scientists' de facto programming environments. Many visualization researchers and practitioners have developed interactive visualization tools that support notebooks, yet little is known about the appropriate design of these tools. To address this critical research gap, we investigate the design strategies in this space by analyzing 163 notebook visualization tools. Our analysis encompasses 64 systems from academic papers and 105 systems sourced from a pool of 55k notebooks containing interactive visualizations that we obtain via scraping 8.6 million notebooks on GitHub. Through this study, we identify key design implications and trade-offs, such as leveraging multimodal data in notebooks as well as balancing the degree of visualization-notebook integration. Furthermore, we provide empirical evidence that tools compatible with more notebook platforms have a greater impact. Finally, we develop SUPERNOVA, an open-source interactive browser to help researchers explore existing notebook visualization tools. SUPERNOVA is publicly accessible at: <https://poloclub.github.io/supernova/>.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '24, May 11–16, 2024, Honolulu, HI, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0331-7/24/05.

<https://doi.org/10.1145/3613905.3650848>

CCS CONCEPTS

• **Human-centered computing** → **Visualization**; **Interactive systems and tools**; **Visualization systems and tools**; **Visualization design and evaluation methods**.

KEYWORDS

Computational Notebook, Interactive Visualization, Systematic Review, Data Science, Design, Cross-Platform Visualization

ACM Reference Format:

Zijie J. Wang, David Munechika, Seongmin Lee, and Duen Horng Chau. 2024. SUPERNOVA: Design Strategies and Opportunities for Interactive Visualization in Computational Notebooks. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3613905.3650848>

1 INTRODUCTION

Computational notebooks, such as Jupyter Notebook [96] and Colab, are the most popular programming environments among data scientists [84]. These notebooks seamlessly combine text, code, and visual outputs in a document that consists of an arbitrary number of *cells*—small text and code editors. Users can execute a code cell, and its output (e.g., text and visualizations) will be displayed below the cell. By providing a literate programming environment, notebooks enable users to perform exploratory data analysis, document their work, and share insights with collaborators [158].

To create easy-to-adopt tools, there is a trend in the VIS community to develop interactive visualization systems that can be used in notebooks [e.g., 135, 197, 207]. Designing visualizations for notebook environments presents unique opportunities and considerations. On the one hand, notebook visualization tools allow direct modification of data through user interactions [179], and users can mix-and-match different visualization tools to create dashboards [192]. However, notebook users often write fragmentary code and execute it nonlinearly [121, 200], which differs from traditional workflows for using interactive visualization systems [35].

Therefore, if researchers do not consider notebooks' unique characteristics, their notebook visualization tools may not fully realize the potential of notebooks and, at worst, may impede the ability of notebook users to effectively use these tools. To shed light on the existing landscape of notebook visualization tools and help visualization researchers and practitioners harness the potential of notebook environments, we **contribute**:

- **The first systematic review of 163 notebook visualization tools** including 64 systems introduced in academic papers and 105 tools sourced from a pool of 55k notebooks containing interactive visualizations that we obtain via scraping 8.6 million notebooks on GitHub (Fig. 2). To inform the design of future tools, we discuss unique design implications (§ 4) and trade-offs (§ 5).
- **Organizational framework to characterize notebook visualization tools** in terms of their motivation for supporting notebooks (§ 4), targeted users (§ 4.1), and a four-dimensional design space based on user needs (§ 5). This framework facilitates a more comprehensive understanding of the landscape of notebook visualization tools. Based on this framework, we further analyze the effects of design factors on the impact of notebook visualization tools. We find tools supporting more notebook platforms have significantly more GitHub stars and paper citations (§ 6).

To broaden the public's access to our collection, we develop SUPERNOVA (Fig. 1), an interactive tool that helps researchers and designers explore existing notebook visualization tools and search for design inspiration and implementation references. Anyone can easily add new tools to this open-source¹ explorer. SUPERNOVA is publicly accessible at: <https://poloclub.github.io/supernova/>.

2 RELATED WORK

Our work joins the research body of studying interactive tools for notebooks. To understand notebook users' behaviors, researchers conduct interview studies [89] and analyze 1 million notebooks scraped from GitHub [158]. Researchers present methods to help researchers *develop* notebook-compatible visualization tools [142, 196]. More recently, a design space analysis is conducted for AI-powered code assistants for notebooks [121]. In contrast, our work focuses on the design of visualization tools for notebooks by analyzing 163 tools identified from academic papers and 8.6 million notebooks. Additionally, inspired by the popular interactive survey browsers for text visualization [100], biological data visualization [88], visualizations for trust in machine learning [33], and embedding visualization [78], we develop SUPERNOVA, the first interactive explorer for notebook visualization tools.

¹SUPERNOVA code: <https://github.com/poloclub/supernova>

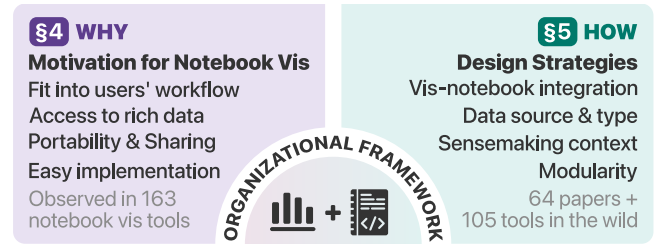


Fig. 2: We present an organizational framework to characterize notebook visualization tools based on their design motivations and strategies through a review of 163 tools.

3 METHODOLOGY

Systematic Review. To study how researchers and practitioners design notebook visualization tools, we collected and analyzed 64 academic papers and 105 tools in the wild. In this study, we define notebook visualization tools as systems that can display interactive visualizations in Python computational notebooks. **(1) Literature collection:** we searched Google Scholar for notebook visualization tools and performed forward and backward reference searches to snowball the results. **(2) In-the-wild tool collection:** we scraped 8.6 million notebooks from GitHub and filtered 55k notebooks containing interactive visualizations by matching notebook cell output types. We extracted 984 potential visualization packages by matching variable names and imported modules using abstract syntax trees, and we manually examined each package to keep 105 that were indeed notebook visualization tools (see § B for details). **(3) Coding:** we conducted a multi-phase coding process to analyze the collected papers, documentation, and demo notebooks. First, three authors independently open coded [28] the same 30 random tools regarding the motivations for using notebooks and design strategies using Google Sheets. After discussing the codebook and resolving disagreements, the three coders independently conducted open coding on the remaining tools, allocating an equal number of tools to each author. Following the analysis of the final codebook and themes, one author applied deductive coding [122] to assign identified design patterns to each tool. We share all scraping code, codebook, and metadata of 163 tools in SUPERNOVA's [repository](#).

Organizational Framework. Our large-scale systematic review resulted in an organizational framework characterizing notebook visualization tools in terms of motivations for supporting notebooks (§ 4), targeted users (§ 4.1), and design patterns based on user needs (§ 5). Using this framework, we develop SUPERNOVA (Fig. 1), an interactive explorer that allows for easy filtering and searching for notebook visualization tools with desired properties. Based on our review, we distill 4 design implications and 4 design trade-offs to help future researchers design notebook visualization tools. Finally, we conduct a correlation analysis and two regression analyses to examine the effects of design patterns on the impacts of notebook interactive visualization tools (§ 6).

4 WHY NOTEBOOK VISUALIZATION TOOLS

This section discusses the motivation for developing interactive visualization tools for computational notebooks. We organize these motivations into four non-mutually exclusive groups.

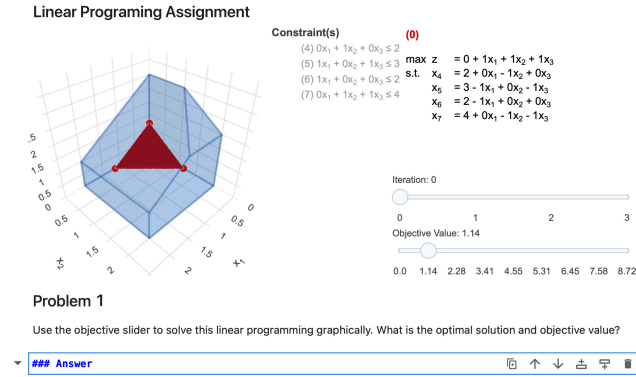


Fig. 3: Many notebook visualization tools are developed for educators and students, such as GILP [152] which offers interactive and easy-to-understand visualizations to help students learn about linear programming algorithms. Educators can directly integrate GILP into notebook-based assignments.

4.1 Seamless Workflow Integration

Our study reveals that most of the surveyed visualization tools support notebooks as a means of aligning with the workflows of end-users. We observe that different user groups have distinct notebook usage patterns. Therefore, to ground our discussion on the notebook workflows of end-users, we categorize end-users into three user groups: data scientists, scientists, and educators and students.

Data Scientists. Notebooks are the most popular programming environment among data scientists [84]. Consequently, many researchers have developed notebook visualization tools to promote adoption among data scientists. Data scientists use notebooks for conducting rapid experiments, collaborating with other stakeholders, and directly deploying notebooks within production pipelines [32]. Notebook visualization tools have covered almost every stage of data scientists' workflow, from annotating data [212] and exploring data [112], to developing ML models [135], documenting models [20], evaluating models [128], and communicating findings to stakeholders [189].

Scientists. Notebooks are also popular among scientists, including biologists and physicists. Scientists use them as an interface for accessing remote clusters [163], and publishing notebooks with academic papers is considered good practice for reproducible research [76]. Thus, many notebook visualization tools are developed to facilitate scientific research workflows, such as designing experiments [70], simulating physical environments [57], and analyzing molecules [131] and astronomical data [8].

Educators and Students. Notebooks are increasingly being used as interactive textbooks in computing education, as they enable students to easily interact with code and test their ideas [173]. Educators also use notebooks for assigning and grading programming assignments [79]. In this use case, notebooks serve as worksheets where students write and run their code in specific cells. We observe a growing trend of notebook visualization tools that are specifically developed for educators and students. For example, GILP [152] visualizes simplex algorithms in notebooks, allowing educators to design interactive textbooks and assignments (Fig. 3).

VizPROG [210] helps instructors monitor students' coding progress during in-class exercises through interactive visualizations.

Our findings highlight that computational notebooks are a popular medium among diverse user groups. In addition to data scientists, scientists, educators, and students also use notebooks in their workflows. This provides visualization researchers and designers with exciting opportunities to develop tools that can be easily adopted. However, we find different user groups have distinct notebook workflows. For example, scientists use notebooks for collaboration and reproducible research, while educators use them as textbooks and worksheets. Therefore, researchers should engage with targeted user groups in the early design process [166] to investigate users' notebook workflows and ground their designs.

Implication on domain-specific design: Designing notebook visualization tools requires researchers to engage with targeted user groups to develop tailored tools, as different user groups have distinct notebook usage patterns.

4.2 Easy Access to Read and Refine Artifacts

Notebook visualization tools not only benefit from easy adoption but also access to programming artifacts, including code, raw data, and models. These tools can be categorized into two groups based on their uses of artifacts.

Artifacts → Visualization Generation. To create visualizations in non-notebook environments, data scientists often need to *manually* specify chart types and input data. However, notebook tools have access to all artifacts needed to create visualizations. For example, B2 [206] uses dataframes and code queries in notebooks to *automatically* synthesize interactive visualizations. Similarly, Lux [109] and SOLAS [50] provide automatic visualization recommendations based on a user's dataframe and analysis history (Fig. 4A). Through accessing ML models that are being trained in notebooks, TENSORBOARD [2] can visualize the model's performance in real time.

Visualizations → Artifact Refinement. After gaining insights from visualizations, data scientists often *manually* refine their code, data, and models outside of notebooks. Notebooks can accelerate this process by directly updating artifacts. For example, MAGE [90] automatically generates code to reflect the change caused by a user's interaction with visualizations (e.g., deleting a column from a table). Similarly, GAM CHANGER [193] enables users to modify ML model weights by direct manipulation on visualizations (Fig. 4B).

When designing notebook visualization tools, it is crucial to consider integrating the *input* and *output* in the visualization workflow [e.g., 31, 35, 180] into the notebook environment. Take Keim et al. [86]'s visual analytics pipeline as an example, the *input data* can be notebook runtime artifacts, text, and usage logs (§ 5.2), and the *output knowledge* can be directly operationalized to synthesize code, transform data, and update ML models in the notebook (§ 5.1).

Implication on new opportunities enabled by easy artifact access: Computational notebooks provide unique opportunities for researchers to integrate the *input* of a visualization pipeline (e.g., notebook runtime artifacts and text) and operationalize its *output* (e.g., transforming data and updating ML models) within the users' existing workflow.

Notebooks Provide Easy Access to User Artifacts

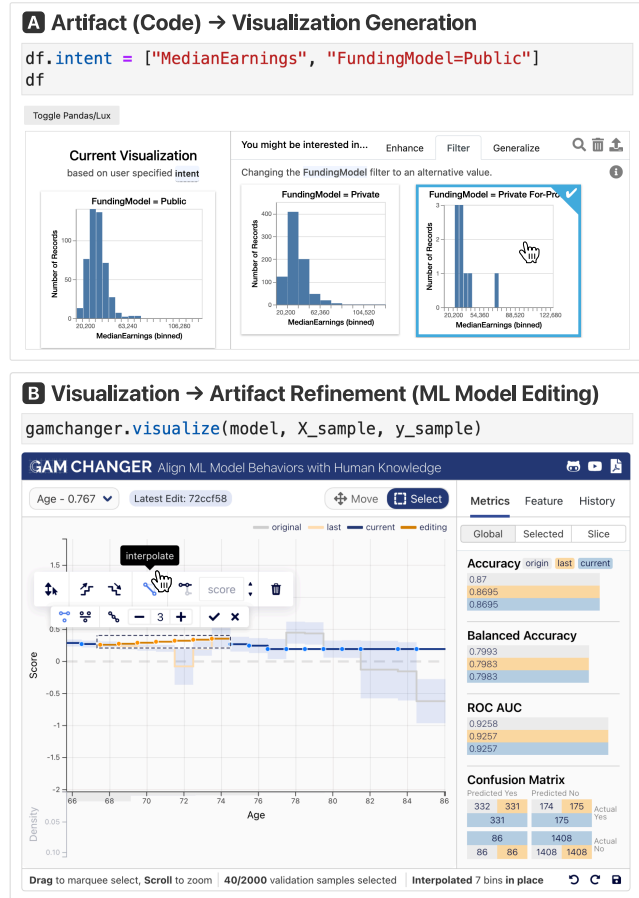


Fig. 4: Computational notebooks offer unique opportunities for visualization tools to *read* and *refine* users' artifacts, such as code, data, and models. For example, (A) Lux [109] leverages a user's data transformation code to recommend visualizations, while (B) GAM CHANGER [193] enables users to interactively edit an ML model's learned weights.

4.3 Portability and Shareability

The notebook community has developed a vibrant ecosystem to convert notebooks into a wide range of mediums. This includes the ability for users to publish notebooks containing interactive visualizations as slides [189], interactive books [38], and dashboards [15]. Therefore, given the portability of notebooks, notebook visualization tools have the potential to reach a more diverse audience. For instance, INTERPRETML [132] leverages Jupyter Book [38] to incorporate in-notebook visualizations into its documentation, providing readers with an engaging way to learn about ML model explanations (Fig. 5). However, different visualization modalities may present unique design challenges, such as potential accessibility concerns for interactive visualizations in presentation slides [208] and the need to consider social contexts for dashboard design [161]. Thus, it is crucial for researchers to carefully consider specific design constraints associated with different modalities if they decide to use notebooks as a bridge to other visualization mediums.

Notebook Visualizations are Easily Portable

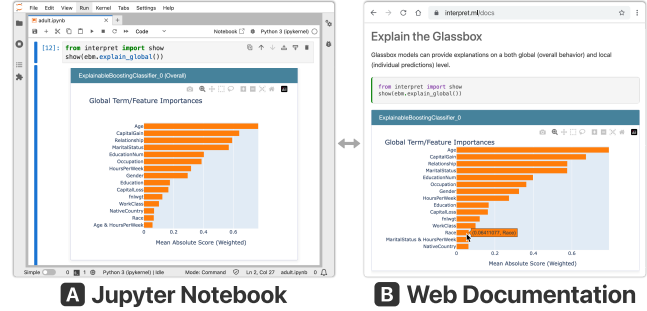


Fig. 5: The vibrant notebook ecosystem enables developers to easily transfer their visualizations across various platforms. For example, (A) the Python library INTERPRETML [194]'s notebook explainable ML visualizations are also used on (B) its documentation website via Jupyter Book [38].

Implication on cross-modality design: The notebook ecosystem offers various options for distributing and sharing notebook visualization tools with diverse stakeholders through various modalities (e.g., interactive books, slides, dashboards). However, researchers need to consider unique design challenges associated with the targeted modalities.

4.4 Ease of Implementation

There exist multiple methods, varying in difficulty, for implementing notebook visualization tools. Some methods are simple and attract researchers to add notebook support for existing visualizations. For example, the ML library CATBOOST [146] uses Jupyter Notebook's native ipywidgets to add checkboxes and sliders to help users customize simple loss function plots. Recent researchers have introduced NOVA workflow [196], which enables easy conversion of web-based visualization apps into notebook widgets [e.g., 128, 193, 197]. Moreover, we observe that some developers use notebooks as a platform for rapidly prototyping and deploying GUI applications. For instance, PIGEON [62] leverages ipywidgets to implement a simple visualization tool that allows annotators to label text and image data. Computational notebooks are web-based systems, and the low barrier to authoring notebook visualization tools reflects and contributes to the trend of web-based interactive visualizations [13, 14]. With the increasing ease of developing notebook visualization tools, we anticipate a growing number of such tools catering to various notebook user groups (§ 4.1).

Implication on growing trend of notebook visualization tools: As the implementation is becoming increasingly accessible, the trend of using computational notebooks as a flexible platform for deploying and developing web-based interactive visualization tools will continue.

5 HOW TO DESIGN NOTEBOOK VIS TOOLS

This section discusses the design patterns of existing notebook visualization tools. To organize these patterns, we construct a four-dimensional design space based on the tool users' needs.

Notebook-Visualization Integration: Data Communication

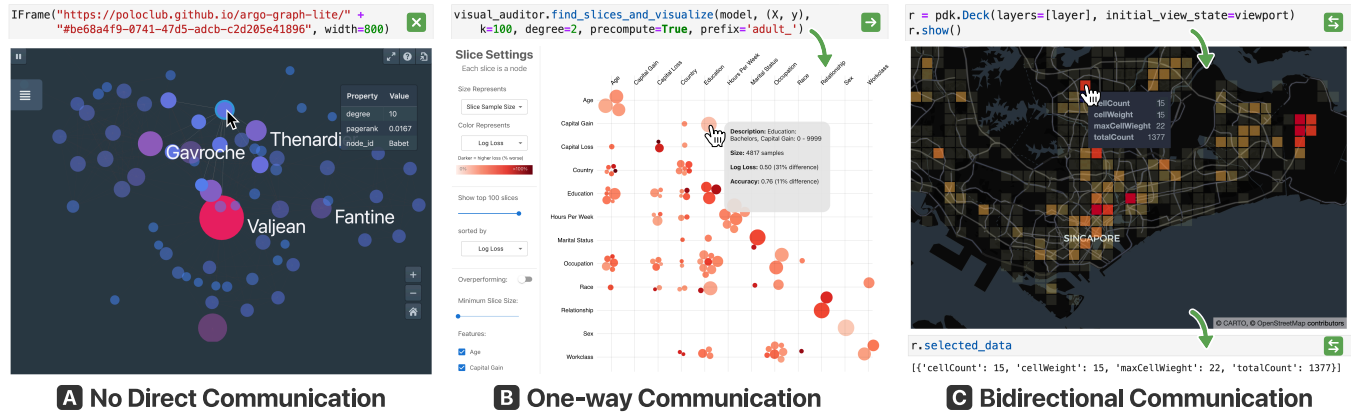


Fig. 6: The integration level between notebook and visualization tools varies based on data communication channels. (A) Tools such as ARGO LITE [111] retrieve data from external servers instead of the notebook. (B) VISUAL AUDITOR [128] visualizes different slices of the dataset that are sent from the notebook. (C) More integrated tools like PYDECK [179] not only visualize data from the notebook but also send data back to the notebook, for example, information on a user's selected map cells.

5.1 Notebook-Visualization Integration

The level of integration between notebook environments and visualization tools can vary widely. We characterize this integration continuum by the *data communication channels* between these two parties, where loosely integrated visualization tools have fewer communication channels than more tightly integrated tools.

✗ No Direct Communication. A few notebook visualization tools do not directly receive data from the notebook environment, as their data source is not available within users' notebooks. Nevertheless, notebooks allow these tools to retrieve data from external sources (§ C.2.1), thereby allowing users to enjoy these tools in their workflows. For example, TENSORBOARD reads log files from the file system, and STATCAST [105] reads data from a separate database server. ARGO LITE [111] allows notebook users to view graph visualizations that are created from a separate website (Fig. 6A).

➔ One-way Communication. Most notebook visualization tools have a one-way communication with the notebook environment: they receive input from the notebook but do not send data back to the notebook (§ C.2.2). (1) Users can *explicitly* specify the input. For example, users can write code to feed an ML model and data into VISUAL AUDITOR [128], which generates interactive visualizations for auditing model biases (Fig. 6B). (2) Some tools also leverage *implicit* input. For instance, SOLAS provides situated visualization recommendations by analyzing a user's historical analysis code. With a one-way communication, users can follow the familiar input-output notebook pattern [96] to customize visualization tools.

↔ Bidirectional Communication. Tools with high notebook integration not only receive input from the notebook but also update its content (§ C.2.3). (1) These tools can *add new code or text* to the notebook. For example, B2 [206] adds a user's interaction history to the notebook cells, and MAGE [90] generates code that can lead to the same consequence as user interactions. (2) Some tools directly *modify the runtime states* in a notebook. For instance, the spatial visualization tool PYDECK [179] stores the user's selected data from the visualization in a runtime variable, which users can access in

other code cells (Fig. 6C). Bidirectional communication in notebooks can be a powerful and unique feature that help interactive visualization users operationalize visualization insights (§ 4.2).

Notebooks enable researchers to integrate both *input* (➔ **one-way communication**) and *output* (↔ **bidirectional communication**) of a visualization pipeline into the users' existing workflow (Implication: new opportunities). However, designing ↔ **bidirectional communication** requires caution. Chattopadhyay et al. [32] find that notebook users often struggle to keep track of the states in different cells. Therefore, automatically modifying notebook states through a visualization tool could cause further confusion. Similarly, in Wu et al. [206]'s study, some participants found it "annoying" when notebook content was populated from a visualization tool. Thus, it is crucial to offer users clear feedback and allow users to configure state-updating behaviors.

Trade-off on data communication: Designing data communication channels (e.g., one-way vs. bidirectional communication) in notebook visualization tools requires careful balance: while bidirectional communication enriches user workflow, it also risks confusion, highlighting the need for clear user feedback and configurable content update policies.

5.2 Data Source and Type

Notebook environments offer rich and multimodal data sources that a visualization tool can use to meet user needs.

📦 Runtime Artifacts. The most common visualization data source is a notebook's runtime artifacts. Visualization tools have access to any data specified by notebook users; existing notebook visualization tools support many data modalities, such as tables [30], spatial data [179], and 3D images [3]. Some tools also leverage ML models in a notebook runtime, helping users interpret transformers [185], curate decision trees [197], calibrate generalized additive models [207], and explore counterfactual explanations [201].

Code and Text. Notebooks combine code and text documentation, which visualization tools can exploit to enhance visualizations (§ C.3). For example, ANTEATER [53] leverages trace-based visualization to help notebook users debug their analysis code. JIGSAW [96] uses variable names in a notebook to validate and correct code generated by AI models. More recently, researchers also use code and text in notebooks to create interactive slides to communicate data insights [110, 189]. Moreover, to help users write high-quality ML model documentation, DocML [20] links a model card visualization to both code and text cells in a notebook.

External Data. Moreover, notebook visualization tools can access data beyond the notebook environment, such as the file system, networks, and hardware information. For example, TENSORBOARD and STATCAST visualize data from a local directory and a database server, respectively. NVDASHBOARD [133] provides notebook users with an interactive dashboard to monitor real-time GPU usage.

Although notebooks provide unique and valuable data for designing interactive visualization tools, accessing various data types requires different implementation strategies. While it is relatively easy to read **runtime artifacts** (§ 4.4), it requires more engineering effort to read **code and text** or implement **bidirectional communication** (see § C for detailed discussion on implementation strategies). Certain strategies are only compatible with specific notebook platforms; for example, tools implemented with Jupyter extensions cannot be used in Google Colab. Thus, there is a trade-off between accessing powerful notebook features and ensuring compatibility with diverse notebook platforms.

Trade-off on compatibility: Notebooks provide access to unique data types, including runtime artifacts, code, text and external data. However, there is a trade-off between leveraging powerful, yet platform-specific features like reading code and text and bidirectional communication, and ensuring broader compatibility across various notebook platforms.

5.3 Display Style & Sensemaking Context

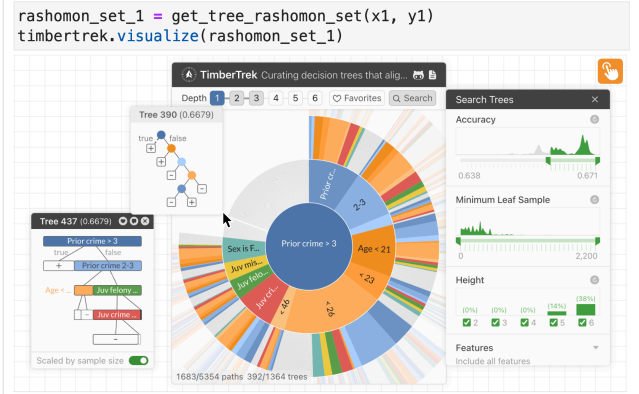
Notebook visualization tools’ display styles can vary based on the user’s sensemaking context [115]. On-demand displays can be used for situational contexts, while always-on displays are suitable for continuous contexts.

On-demand display. Most visualization tools show visualizations below a code cell [e.g., 193, 201, 207]. These visualizations are part of the cell flow—they move vertically with the cells when a user scrolls through the notebook. With this layout, users can easily create multiple instances of the same visualization tool with different input data. For example, users can create multiple instances of TIMBERTREK [197] in different cells with different collections of decision trees and compare across these collections (Fig. 7A).

Always-on display. Notebook tools can also display visualizations outside of notebook cells (§ C.4), leading to an always-on display detached from the cell flow. For instance, AUTOPROFILER [206] continuously updates data distribution visualizations in a resizable dashboard pane to the right of the notebook UI, allowing users to view persistent data profiling information while exploring their datasets (Fig. 7B). Similarly, NVDASHBOARD [133] displays multiple

Sensemaking Context of Notebook Visualizations

A On-demand Display



B Always-on Display

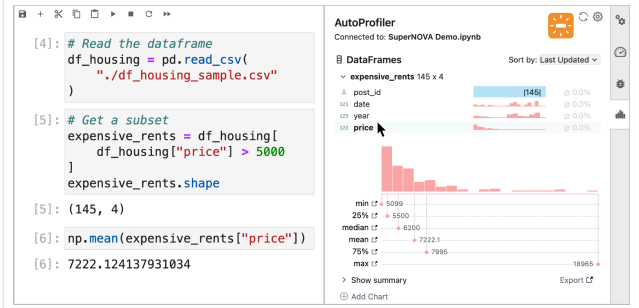


Fig. 7: Notebook visualization tools’ display styles vary based on the user’s sensemaking context. For example, (A) TIMBERTREK [197] uses an on-demand display to visualize a large collection of decision trees next to the cells where the trees are created. (B) AUTOPROFILER [49] leverages an always-on display to automatically and continuously highlight data distributions and summary statistics of the user’s datasets.

charts outside of the notebook UI so that users can monitor their GPU usage in real time while interacting with the notebook.

The design choice of visualization display style in the notebook depends on the users’ needs and the sensemaking context. Based on Liu and Stasko [115]’s sensemaking model, visualizations provide external anchoring, cognitive offloading and information foraging. They suggest that visualization designers should minimize the “semantic distance” [81] between the tasks users want to perform and the physical form of visualizations. In computational notebooks, an **on-demand display** can assist users with situational sensemaking and temporary anchoring for comparisons. On the other hand, an **always-on display** can be beneficial for ongoing monitoring and tasks that require continuous cognitive offloading.

Trade-off on display style: Researchers need to consider the trade-off between on-demand and always-on displays of interactive visualizations in notebooks based on the users’ needs. On-demand displays aid situational sensemaking and comparisons, while always-on displays support continuous monitoring and cognitive offloading.

5.4 Modularity

Modularity in notebook visualization tools is a critical consideration when catering to different analysis needs, such as exploratory and exploitative [12], and user’s programming proficiency. This ensures a balance between the code and the graphical user interface.

Monolithic System. Most notebook visualization tools are monolithic, presenting the entire system all at once. For example, when a user calls `YDATA-PROFILLING` [30] in a notebook cell, the tool displays a panel beneath the cell that contains all exploratory data analysis visualizations (Fig. 8A). These visualizations are organized into multiple tabs based on their tasks, such as variable interactions, correlations, and missing values.

Modular Components. Modular visualization tools accommodate the fragmentary nature of notebook code and allow users to easily *customize* and *compose* visualizations. For example, `AEQUITAS` [159], an ML fairness auditing toolkit, provides different interactive visualizations for different fairness metrics. These visualizations are modularized into separate functions, enabling users to write code to generate and compose visualizations that meet specific needs (Fig. 8B). For instance, with `AEQUITAS`, a user can create and inspect a fairness overview in a notebook cell and delve into specific fairness metrics in other separate cells.

Monolithic and modular architectures have been extensively discussed in the software engineering literature for decades [6]. Within the visual analytics research community, there is a recent trend towards shifting from designing “over-complicated” monolithic systems to simpler and reusable modular components [18, 204]. The use of **modular components** aligns well with computational notebooks, as notebook users can easily display and customize different components in separate notebook cells. Additionally, users can take advantage of dashboard authoring tools [15, 192] to compose different visualization components into a dashboard directly in their notebooks (§ 4.3). However, **modular components** require the users to know their visualization goals (i.e., *exploitative* analysis) and know how to write code to display the appropriate components. In contrast, a **monolithic system** is more friendly to beginner users and suitable for *exploratory* analysis, where it can guide users to uncover data patterns and insights.

Trade-off on modularity: Modular visualization tools are composable and reusable, particularly in notebooks where users can easily display and customize them. While modular components offer flexibility for users with clear analysis goals and coding skills, monolithic systems remain more beginner-friendly and ideal for exploratory analysis.

6 ANALYSIS

Leveraging our organizational framework as a lens, we conduct a quantitative analysis to study the relationship between the design of notebook visualization tools and the impacts of these tools (e.g., GitHub star count and publication citation count). Our analysis offers additional insights into future design decisions.

Data Collection. We characterize all 163 notebook visualization tools using our framework (Table 1). Then, we collect the GitHub star count, first commit date, publication year, and citation count of

Modularity of Notebook Visualizations

A Monolithic System

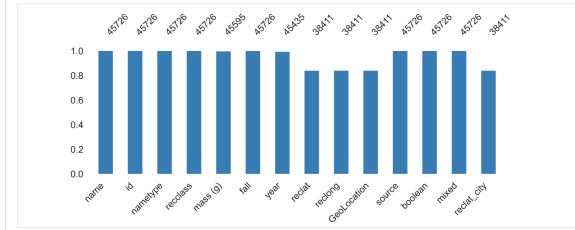
```
df.profile_report(
    sort=None,
    html={"style": {"full_width": True}}, progress_bar=False
)
```

Pandas Profiling Report Overview Variables Interactions Missing values Sample

Overview

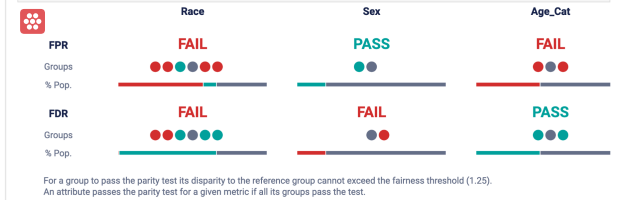
Dataset statistics		Variable types	
Number of variables	14	Text	3
Number of observations	45726	Numeric	5
Missing cells	29682	Categorical	4
Missing cells (%)	4.6%	DateTime	1
Duplicate rows	0	Boolean	1
Duplicate rows (%)	0.0%		
Total size in memory	4.6 MiB		
Average record size in memory	105.0 B		

Missing values



B Modular Components

```
audit.summary_plot(metrics=metrics,
    fairness_threshold=disparity_tolerance)
```



```
audit.disparity_plot(metrics=metrics,
    attribute='race', fairness_threshold=disparity_tolerance)
```

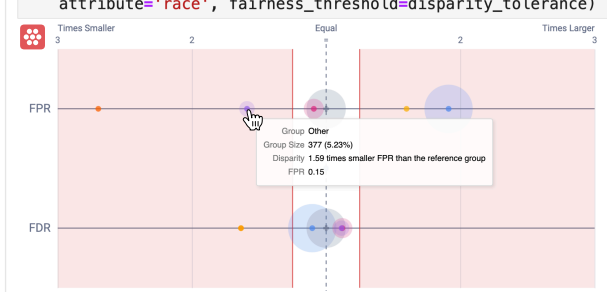


Fig. 8: The modularity of notebook visualization tools varies. For example, (A) `YDATA-PROFILLING` [30] shows all components in a notebook cell, where users navigate data profiling visualizations via tabs. (B) In contrast, `AEQUITAS` [159] modularizes different visualization components into distinct Python functions, enabling users to write code to show ML model fairness visualizations tailored to specific needs.

all available tools via the GitHub API [63] and Semantic Scholar API [93]. Among all 163 tools, 135 have GitHub repositories and 76 have Semantic Scholar entries.

Tools that support more notebook platforms have significantly higher impacts

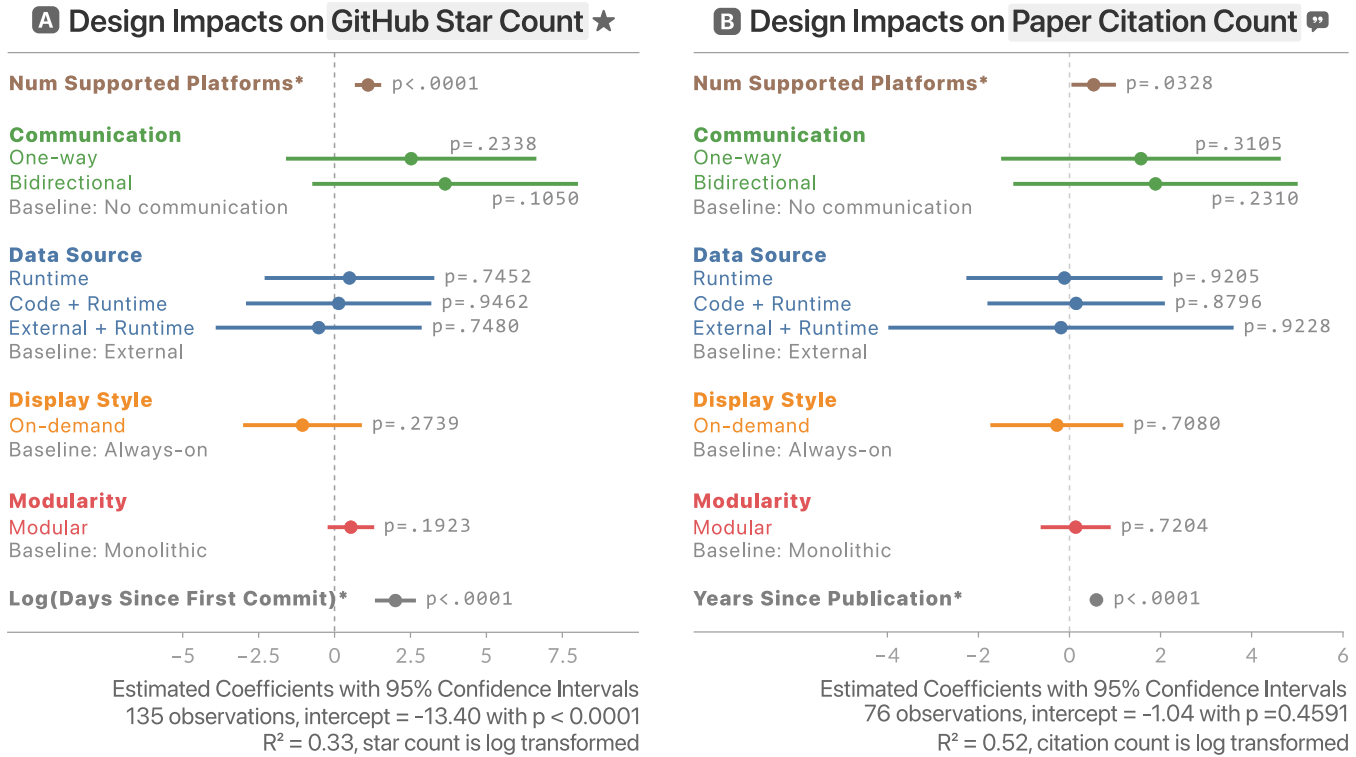


Fig. 9: We conduct two regression analyses to investigate the effects of various design factors on the impact of notebook visualization tools, as measured by (A) GitHub star count and (B) paper citation count. We encode categorical variables using dummy variables, with the baseline category labeled in the figure. The results reveal that, in addition to time, notebook interactive visualization tools that support more notebook platforms have significantly more GitHub stars and paper citations.

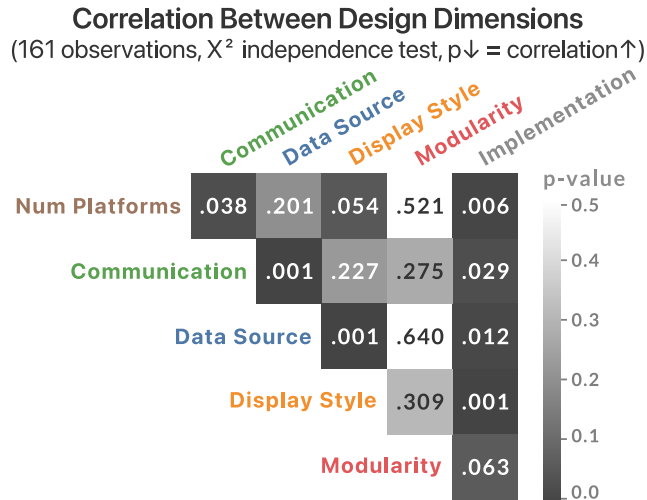


Fig. 10: Design dimension correlations via pair-wise X^2 tests.

Correlation Analysis. We analyze the correlations across different design dimensions by conducting pair-wise X^2 independence tests (Fig. 10). Unsurprisingly, our results highlight that **implementation strategies** correlate with many other design dimensions. For example, tools that support **always-on display** are more

likely to be **implemented using extensions**. Interestingly, **data source** is correlated with both **communication** and **display** styles. In particular, tools that access **code and text** from the notebook are more likely to support **bidirectional communication** and **always-on display**. We hypothesize that this is because designers often use text and code from a notebook for generative tasks (e.g., automatic visualization generation), and they prefer always-on displays to provide notebook users with continuous feedback (**Trade-off: display style**). For example, visualization recommendation tools B2 [206] and PI2 [36] leverage existing code and text in a notebook to generate new visualization code in the notebook and display synthesized visualizations on an always-on panel. Finally, we observe that tools with **bidirectional communication** support much less **notebook platforms** than tools with **one-way communication**. This empirical finding reflects the trade-off between notebook integration and platform dependency (**Trade-off: compatibility**).

Regression Analysis. We conduct two regression analyses to examine the effects of design factors on the impact of notebook visualization tools, as measured by GitHub star and paper citation counts (Fig. 9). Since **implementation strategies** correlate with many other design dimensions, we do not include it in both regression models. We include time as an independent variable and use dummy variables to encode categorical variables. The results

highlight that tools **supporting more notebook platforms** have significantly more GitHub stars and paper citations. Other design dimensions do not significantly affect the popularity and recognition of notebook visualization tools. This result implies that future researchers and developers should prioritize notebook platform compatibility to maximize the impact of their tools.

7 DISCUSSION AND FUTURE WORK

By analyzing 163 interactive notebook visualization tools identified from 8.6 million public notebooks and 64 academic papers (§ 3), we present an organization framework to characterize these tools (§ 4, § 5). We provide practice design implications and trade-offs as well as insights from statistical analyses (§ 6). Based on our findings, we discuss future research opportunities and limitations of our study.

Democratizing Notebook Visualization Tool Creation. We discover a spectrum of methods, varying in difficulty, for authoring notebook visualization tools (§ 4.4). In particular, accessing **code and text** and supporting **bidirectional communication** require significant engineering effort (§ 5.2). Furthermore, some implementation strategies are only compatible with specific notebook platforms (**Trade-off: compatibility**). Therefore, we see research opportunities to lower the barrier to authoring notebook interactive visualization tools that harness the full potential of notebook platforms. First, practitioners often use libraries such as D3 [24] and VegaLite [162] to develop web-based interactive visualizations. It would be valuable if these libraries integrated native support for notebook platforms or new libraries specifically targeted authoring notebook visualizations. On the other hand, researchers can also enhance notebook platforms to better support interactive visualizations. For example, similar to browser vendors sharing the same web standard, researchers can develop a universal notebook protocol that enables developers to access and communicate data using a standardized method across notebook platforms.

Enriching Fluid Notebook-Vis Integration. The design trade-offs regarding visualization display styles (**Trade-off: display style**) and modularity (**Trade-off: modularity**) partially arise from the rigid layout of the popular cell-based notebooks [107]. For example, most notebook platforms present cells in a linear manner, thereby requiring designers to decide whether to display their visualization tools within the flow (**on-demand display**) of the cell or detach them from the flow (**always-on display**). To address this trade-off, researchers can explore alternative notebook layouts. For example, researchers have introduced sticky cells [192] to break the linear presentation of notebook cells. These sticky cells provide visualization designers with the flexibility to seamlessly switch between **on-demand** and **always-on** displays (Fig. 11). Similarly, regarding the modularity of visualization tools, future researchers could develop intelligent notebook interfaces that automatically adapt a visualization tool between **modular** and **monolithic** modes based on the users' current tasks and requirements.

Promoting Responsible AI through Notebook Workflows. We observe an interesting trend that researchers exploit notebooks as a means to promote responsible AI practices (e.g., AEQUITAS [159], FAIRLEARN [45], FARSIGHT [195], and MLDOC [20]). We identify two motivations for this emerging trend. First, AI practitioners often lack incentives to adopt responsible AI practices [151, 164],

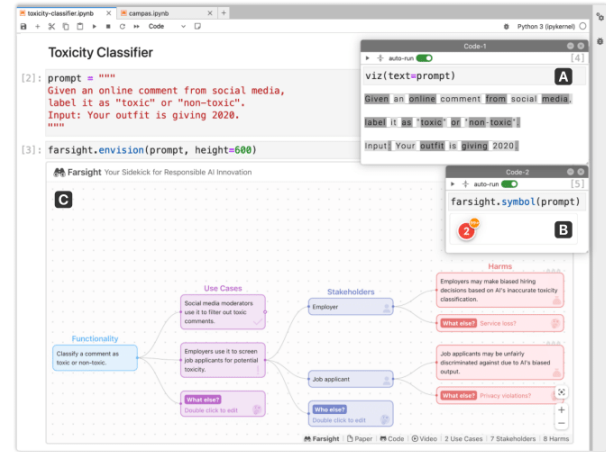


Fig. 11: STICKYLAND [192] enables a non-linear notebook layout, allowing notebook users to easily switch between on-demand and always-on displays of notebook visualization tools. For example, while prototyping large language model-powered apps by writing prompts, (A) a user can create a sticky cell with HUGGINGFACE TOKENIZER [127] to continuously visualize the prompt’s tokenization patterns. (B) The user can also use an always-on display with FARSIGHT [195] to alert to potential risks associated with the prompt. (C) Moreover, the user can use FARSIGHT’s interactive tree visualization in an on-demand display for brainstorming use cases, stakeholders, and potential harms of their apps.

such as fairness assessment and model documentation. By integrating responsible AI practices directly into practitioners’ existing notebook workflows (§ 4.1), researchers aim to minimize adoption friction and “nudge” [20] practitioners to follow these practices. For example, FARSIGHT alerts users to potential harms of their large language model-powered apps while they are developing prompts in a notebook (Fig. 11). Similarly, MLDOC automatically creates and shows an AI “model card” [126] using content from a notebook.

Secondly, responsible AI requires collaboration across disciplines and teams within an organization [151, 190]. Because AI practitioners have already been using notebooks to collaborate with diverse stakeholders (e.g., designers and managers) [211], researchers leverage notebooks as a boundary object to facilitate responsible AI practices across teams. For example, in Deng et al. [43]’s study on ML fairness toolkits, a participant highlighted “a simple notebook format and compelling visualizations are needed for [organizational] leadership to adopt the toolkits.” Thus, as the mitigation of AI harms has become increasingly crucial, we see exciting research opportunities for researchers to design, develop, and evaluate notebook visualization tools to promote responsible AI.

Limitations. In this study, to keep our review manageable and focused, we focus on computational notebooks designed for Python, the most commonly used programming language among data scientists [84]. Future work can explore notebooks designed for other languages, such as R Markdown [176] for R and Observable [134] for JavaScript. As notebook visualization tools are still nascent, there are limited user studies evaluating the effectiveness of these

tools. In addition, although there are many different notebook user groups (§ 4.1), the existing HCI notebook research focuses on data scientists [107]. To broaden the understanding of notebook visualization tools, future research endeavors can involve engaging with diverse user groups, including scientists, educators, students, and users with accessibility needs.

8 CONCLUSION

We collect a total of 163 notebook visualization tools, including 64 from academic papers and 103 sourced from a pool of 55k notebooks containing interactive visualizations that we obtain by scraping 8.6 million notebooks on GitHub. Based on our review, we introduce a framework for characterizing these tools in terms of their motivation for supporting notebooks, targeted users, and design patterns. We further discuss key design implications and trade-offs as well as research opportunities for notebook visualization. Finally, we present SUPERNOVA to help researchers and developers easily explore existing notebook visualization tools. We hope that our work contributes to a more comprehensive understanding of notebook visualization tools and helps researchers design and develop visualization tools that are easy to use and adopt.

ACKNOWLEDGMENTS

This work was supported by a J.P. Morgan PhD Fellowship, Apple Scholars in AI/ML PhD fellowship, gifts from Bosch and Cisco. We thank anonymous reviewers for their valuable feedback.

REFERENCES

- [1] AaltoGIS. 2020. Spatial Data Science for Sustainable Development. AaltoGIS. <https://github.com/AaltoGIS/Sustainability-GIS>
- [2] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *OSDI*. <https://dl.acm.org/doi/10.5555/3026877.3026899>
- [3] Alexandre Abraham, Fabian Pedregosa, Michael Eickenberg, Philippe Gervais, Andreas Mueller, Jean Kossaifi, Alexandre Gramfort, Bertrand Thirion, and Gaël Varoquaux. 2014. Machine Learning for Neuroimaging with Scikit-Learn. *Front. Neuroinform* (2014). <https://doi.org/10.3389/fninf.2014.00014>
- [4] Evidently AI. 2022. Evidently: Evaluate and Monitor ML Models from Validation to Production. Evidently AI. <https://github.com/evidentlyai/evidently>
- [5] Eugenio Angriman, Fabian Brandt-Tumescheit, Leon Franke, Alexander van der Grinten, and Henning Meyerhenke. 2022. Interactive Visualization of Protein RINs Using NetworkKit in the Cloud. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. <https://doi.org/10.1109/IPDPSW55747.2022.00055>
- [6] M. Aoyama. 1998. Agile Software Process and Its Experience. In *Proceedings of the 20th International Conference on Software Engineering*. <https://doi.org/10.1109/ICSE.1998.671097>
- [7] Apache. 2019. Apache Beam: Unified Programming Model for Batch and Streaming Data Processing. The Apache Software Foundation. <https://github.com/apache/beam>
- [8] M. Araya, M. Osorio, M. Diaz, C. Ponce, M. Villanueva, C. Valenzuela, and M. Solar. 2018. JOVIAL: Notebook-based Astronomical Data Analysis in the Cloud. *Astronomy and Computing* 25 (2018). <https://doi.org/10.1016/j.ascom.2018.09.001>
- [9] Ran Aroussi. 2019. Quantstats: Portfolio Analytics for Quants, Written in Python. <https://github.com/ranaroussi/quantstats>
- [10] Autodesk. 2016. Notebook Molecular Visualization. <https://github.com/Autodesk/notebook-molecular-visualization>
- [11] AutoViML. 2020. AutoViz: Automatically Visualize Any Dataset, Any Size with a Single Line of Code. <https://github.com/AutoViML/AutoViz>
- [12] Andrea Batch and Niklas Elmquist. 2018. The Interactive Visualization Gap in Initial Exploratory Data Analysis. *IEEE Transactions on Visualization and Computer Graphics* 24 (2018). <https://doi.org/10.1109/TVCG.2017.2743990>
- [13] Leilani Battle, Peitong Duan, Zachery Miranda, Dana Mukusheva, Remco Chang, and Michael Stonebraker. 2018. Beagle: Automated Extraction and Interpretation of Visualizations from the Web. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3173574.3174168>
- [14] Leilani Battle, Danni Feng, and Kelli Webber. 2022. Exploring D3 Implementation Challenges on Stack Overflow. In *2022 IEEE Visualization Conference (VIS)*. *arXiv 2108.02299*. <http://arxiv.org/abs/2108.02299>
- [15] Alex Bäuerle, Ángel Alexander Cabrera, Fred Hohman, Megan Maher, David Koski, Xavier Suau, Titus Barik, and Dominik Moritz. 2022. Symphony: Composing Interactive Interfaces for Machine Learning. In *CHI*. <https://doi.org/10.1145/3491102.3502102>
- [16] Antoni Baum. 2020. PyCaret: An Open-Source, Low-Code Machine Learning Library in Python. PyCaret. <https://github.com/pycaret/pycaret>
- [17] Rohan Bavishi, Shadaj Laddad, Hiroaki Yoshida, Mukul R. Prasad, and Koushik Sen. 2021. VizSmith: Automated Visualization Synthesis by Mining Data-Science Notebooks. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. <https://doi.org/10.1109/ASE51524.2021.9678696>
- [18] Enrico Bertini. 2022. Building (Easy-To-Adopt) Software While Doing Visualization Research. <https://filwd.substack.com/p/building-easy-to-adopt-software-while>
- [19] Francois Bertrand. 2020. SweetViz: In-depth EDA in Two Lines of Code. <https://github.com/fbdesignpro/sweetviz>
- [20] Avinash Bhat, Austin Coursey, Grace Hu, Sixian Li, Nadia Nahar, Shurui Zhou, Christian Kästner, and Jin L. C. Guo. 2023. Aspirations and Practice of Model Documentation: Moving the Needle with Nudging and Traceability. In *CHI*. *arXiv 2204.06425*. <https://doi.org/10.1145/3544548.3581518>
- [21] Bloomberg. 2019. Ipydatagrid: Fast Datagrid Widget for the Jupyter Notebook and JupyterLab. Bloomberg. <https://github.com/bloomberg/ipydatagrid>
- [22] Bokeh Development Team. 2014. Bokeh: Python Library for Interactive Visualization. <http://www.bokeh.pydata.org>
- [23] Centre Borelli. 2019. Pypotree: Potree for Jupyter Notebooks and Colab. <https://github.com/centreborelli/pypotree>
- [24] Michael Bostock, Vadim Ogievetsky, and Jeffrey Heer. 2011. D³ Data-Driven Documents. *IEEE TVCG* 17 (2011). <https://doi.org/10.1109/TVCG.2011.185>
- [25] Jorge Boucas. 2015. Py2cytoscape: Python Utilities for Cytoscape and Cytoscape.js. Cytoscape Consortium. <https://github.com/cytoscape/py2cytoscape>
- [26] Cédric Bouysset. 2021. Mols2grid - Interactive Molecule Viewer for 2D Structures. <https://doi.org/10.5281/zenodo.6591473>
- [27] Bqplot. 2016. Bqplot: Plotting Library for IPython/Jupyter Notebooks. <https://github.com/bqplot/bqplot>
- [28] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3 (2006). <https://doi.org/10.1191/1478088706qp0630a>
- [29] M. A. Bredtels. 2016. Interactive (Statistical) Visualisation and Exploration of a Billion Objects with Vaex. *Proceedings of the International Astronomical Union* 12 (2016). <https://doi.org/10.1017/S1743921316012795>
- [30] Simon Brugman. 2019. Pandas-Profilng: Exploratory Data Analysis. <https://github.com/pandas-profilng/pandas-profilng>
- [31] Dylan Cashman, Shah Rukh Humayoun, Florian Heimerl, Kendall Park, Subhajit Das, John Thompson, Bahador Saket, Abigail Mosca, John Stasko, Alex Ender, Michael Gleicher, and Remco Chang. 2019. A User-based Visual Analytics Workflow for Exploratory Model Analysis. *Computer Graphics Forum* 38 (2019). <https://doi.org/10.1111/cgf.13681>
- [32] Souti Chattopadhyay, Ishita Prasad, Austin Z. Henley, Anita Sarma, and Titus Barik. 2020. What's Wrong with Computational Notebooks? Pain Points, Needs, and Design Opportunities. In *CHI*. <https://doi.org/10.1145/3313831.3376729>
- [33] Angelos Chatzimparmpas, Kostiantyn Kucher, and Andreas Kerren. 2024. Visualization for Trust in Machine Learning Revisited: The State of the Field in 2023. *IEEE Computer Graphics and Applications* (2024). <https://doi.org/10.1109/MCG.2024.3360881>
- [34] Taher Chegini, Hong-Yi Li, and L. Ruby Leung. 2021. HyRiver: Hydroclimate Data Retriever. *Journal of Open Source Software* 6 (2021). <https://doi.org/10.21105/joss.03175>
- [35] Min Chen and Amos Golan. 2016. What May Visualization Processes Optimize? *IEEE Transactions on Visualization and Computer Graphics* 22 (2016). <https://doi.org/10.1109/TVCG.2015.2513410>
- [36] Yiru Chen and Eugene Wu. 2022. PI2: End-to-end Interactive Visualization Interface Generation from Queries. In *Proceedings of the 2022 International Conference on Management of Data*. <https://doi.org/10.1145/3514221.3526166>
- [37] François Chollet. 2015. Keras. (2015). <https://keras.io>
- [38] Executable Books Community. 2020. Jupyter Book. Zenodo. <https://doi.org/10.5281/ZENODO.4539666>
- [39] Damon Crockett. 2021. Ipy: Iconographic Visualization Inside Computational Notebooks. *International Journal for Digital Art History* (2021). <https://doi.org/10.11588/DAH.2019.4.66401>
- [40] Cuemacro. 2016. Chartpy: Easy to Use Python API Wrapper to Plot Charts with Matplotlib, Plotly, Bokeh and More. <https://github.com/cuemacro/chartpy>
- [41] Datapane. 2023. Datapane: Build Full-Stack Data Apps in 100% Python. Datapane. <https://github.com/datapane/datapane>
- [42] Dawson-Haggerty et al. 2019. Trimesh. <https://github.com/mikedh/trimesh>

- [43] Wesley Hanwen Deng, Manish Nagireddy, Michelle Seng Ah Lee, Jatinder Singh, Zhiwei Steven Wu, Kenneth Holstein, and Haiyi Zhu. 2022. Exploring How Machine Learning Practitioners (Try To) Use Fairness Toolkits. In *2022 ACM Conference on Fairness, Accountability, and Transparency*. <https://doi.org/10.1145/3531146.3533113>
- [44] Ian Drosos, Titus Barik, Philip J. Guo, Robert DeLine, and Sumit Gulwani. 2020. Wrex: A Unified Programming-by-Example Interaction for Synthesizing Readable Code for Data Scientists. In *CHI*. <https://doi.org/10.1145/3313831.3376442>
- [45] Miro Dudík, Sarah Bird, Hanna Wallach, and Kathleen Walker. 2020. Fairlearn: A Toolkit for Assessing and Improving Fairness in AI. (2020). <https://www.microsoft.com/en-us/research/publication/fairlearn-a-toolkit-for-assessing-and-improving-fairness-in-ai/>
- [46] xavier dupré. 2016. Jyquickhelper: Helpers for Jupyter Notebooks around Javascript. <https://github.com/sdpython/jyquickhelper>
- [47] Martin Durant. 2018. Intake: A General Interface for Loading Data. Intake. <https://github.com/intake/intake>
- [48] Enthought. 2015. Mayavi: 3D Visualization of Scientific Data in Python. Enthought, Inc.. <https://github.com/enthought/mayavi>
- [49] Will Epperson, Vaishnavi Gorantla, Dominik Moritz, and Adam Perer. 2023. Dead or Alive: Continuous Data Profiling for Interactive Data Science. *IEEE Transactions on Visualization and Computer Graphics* (2023). <https://doi.org/10.1109/TVCG.2023.3327367>
- [50] Will Epperson, Doris Jung-Lin Lee, Leijie Wang, Kunal Agarwal, Aditya G. Parameswaran, Dominik Moritz, and Adam Perer. 2022. Leveraging Analysis History for Improved In Situ Visualization Recommendation. *Computer Graphics Forum* 41 (2022). <https://doi.org/10.1111/cgf.14529>
- [51] Facebook. 2019. Ax: Adaptive Experimentation Platform. Meta. <https://github.com/facebook/Ax>
- [52] Facebook. 2020. HiPlot Makes Understanding High Dimensional Data Easy. <https://github.com/facebookresearch/hiplot>
- [53] Rebecca Faust, Carlos Scheidegger, Katherine Isaacs, William Z. Bernstein, Michael Sharp, and Chris North. 2022. Interactive Visualization for Data Science Scripts. In *2022 IEEE Visualization in Data Science (VDS)*. <https://doi.org/10.1109/VDS57266.2022.00009>
- [54] Filipe Fernandes. 2019. Folium: Python Data. Leaflet.js Maps. <https://github.com/python-visualization/folium>
- [55] Nicolas F. Fernandez, Gregory W. Gundersen, Adeeb Rahman, Mark L. Grimes, Klarisa Rikova, Peter Hornbeck, and Avi Ma'ayan. 2017. Clustergrammer, a Web-Based Heatmap Visualization and Analysis Tool for High-Dimensional Biological Data. *Scientific Data* 4 (2017). <https://doi.org/10.1038/sdata.2017.151>
- [56] Max Franz, Manfred Cheung, Onur Sumer, Gerardo Huck, Dylan Fong, R-Ba, Josejulio Martinez, Jan Žák, Tony Mullen, Bogdan Chadkin, Ayhun, Metincan-siper, Chris, Jan Hartmann, Joseph Stahl, Paolo Parlapiano, Eli Sherer, Mélanie Gauthier, Rich Trott, Yaroslav Sidlovsky, Bumbui, Alexander Li, Christian Lopes, TexKiller, Mike Beynon, Gui Meira, Janit Mehta, and Mike Dias. 2022. Cytoscape/Cytoscape.js. Zenodo. <https://doi.org/10.5281/ZENODO.6828253>
- [57] C. Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. 2021. Brax: A Differentiable Physics Engine. <http://github.com/google/brax>
- [58] Takanori Fujiwara, Xinhai Wei, Jian Zhao, and Kwan-Liu Ma. 2022. Interactive Dimensionality Reduction for Comparative Analysis. *IEEE Transactions on Visualization and Computer Graphics* 28 (2022). <https://doi.org/10.1109/TVCG.2021.3114807>
- [59] Patrick Fuller. 2013. Imolecule: An Embeddable webGL Molecule Viewer and File Format Converter. <https://github.com/patrickfuller/imolecule>
- [60] Patrick Fuller. 2013. Jgraph: An Embeddable webGL Graph Visualization Library. <https://github.com/patrickfuller/jgraph>
- [61] Katarina Furmanova, Samuel Gratzl, Holger Stitz, Thomas Zichner, Miroslava Jaresova, Alexander Lex, and Marc Streit. 2020. Taggle: Combining Overview and Details in Tabular Data Visualizations. *Information Visualization* 19 (2020). <https://doi.org/10.1177/1473871619878085>
- [62] Anastasis Germanidis. 2017. Pigeon: Quickly Annotate Data on Jupyter. <https://github.com/agermanidis/pigeon>
- [63] GitHub. 2023. GitHub GraphQL API Documentation. <https://ghdocs-prod.azurewebsites.net/en/graphql>
- [64] Carlos Gonzalez. 2019. Hciplot: Library for Visualizing High-Contrast Imaging Multidimensional Datacubes on JupyterLab. <https://github.com/carlos-gg/hciplot>
- [65] Google. 2018. TensorFlow Model Analysis. <https://github.com/tensorflow/model-analysis>
- [66] Google. 2021. Brax: Massively Parallel RigidBody Physics Simulation on Accelerator Hardware. <https://github.com/google/brax>
- [67] Graphistry. 2016. PyGraphistry: Explore Relationships. <https://github.com/graphistry/pygraphistry>
- [68] Anita Graser and Melitta Dragaschnig. 2020. Exploring Movement Data in Notebook Environments. In *IEEE VIS 2020 Workshop on Information Visualization of Geospatial Networks, Flows and Movement (MoVis)*. http://move.geog.ucsb.edu/wp-content/uploads/2020/10/MoVIS20_paper_4.pdf
- [69] Maarten Grootendorst. 2022. BERTopic: Neural Topic Modeling with a Class-Based TF-IDF Procedure. *arXiv preprint arXiv:2203.05794* (2022). <https://doi.org/10.48550/arXiv.2203.05794>
- [70] Grace Guo, Maria Glenski, ZhuanYi Shaw, Emily Saldanha, Alex Endert, Svitlana Volkova, and Dustin Arendt. 2021. Vaine: Visualization and AI for Natural Experiments. In *2021 IEEE Visualization Conference (VIS)*. <https://doi.org/10.1109/VIS49827.2021.9623285>
- [71] Grace Guo, Ehud Karavani, Alex Endert, and Bum Chul Kwon. 2023. Causalvis: Visualizations for Causal Inference. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3544548.3581236>
- [72] Abhishek Gupta. 2021. Data-Purifier: A Python Library for Automated Exploratory Data Analysis. <https://github.com/Elysian01/Data-Purifier>
- [73] Alexander B. Gurvich and Aaron M. Geller. 2023. Firefly: A Browser-based Interactive 3D Data Visualization Tool for Millions of Data Points. *The Astrophysical Journal Supplement Series* 265 (2023). <https://doi.org/10.3847/1538-4365/acb59f>
- [74] Robert Haas. 2021. Gravis: Interactive Graph Visualizations with Python and HTML/CSS/JS. <https://github.com/robert-haas/gravis>
- [75] Jürgen Hackl. 2019. Pathpy: An OpenSource Python Package for the Analysis of Time Series Data on Networks Using Higher-Order and Multi-Order Graphical Models. <https://github.com/pathpy/pathpy>
- [76] Falk Herwig, Robert Andrassy, Nic Annau, Ondrea Clarkson, Benoit Côté, Aaron D'Sa, Sam Jones, Belaid Moa, Jericho O'Connell, David Porter, Christian Ritter, and Paul Woodward. 2018. Cyberhubs: Virtual Research Environments for Astronomy. *The Astrophysical Journal Supplement Series* 236 (2018). <https://doi.org/10.3847/1538-4365/aab777>
- [77] Patrik Hlobil. 2018. Pandas-Bokeh: Bokeh Plotting Backend for Pandas and GeoPandas. <https://github.com/PatrikHlobil/Pandas-Bokeh>
- [78] Z. Huang, D. Witschard, K. Kucher, and A. Kerren. 2023. VA + Embeddings STAR: A State-of-the-Art Report on the Use of Embeddings in Visual Analytics. *Computer Graphics Forum* 42 (2023). <https://doi.org/10.1111/cgf.14859>
- [79] Matthew Hull, Vivian Pednekar, Hannah Murray, Nimisha Roy, Emmanuel Tung, Susanta Routray, Connor Guerin, Justin Chen, Zijie J. Wang, Seongmin Lee, Mahdi Roozbahani, and Duen Horng Chau. 2023. VISGRADER: Automatic Grading of D3 Visualizations. *IEEE Transactions on Visualization and Computer Graphics* (2023). <https://doi.org/10.1109/TVCG.2023.3327181>
- [80] J. D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9 (2007). <https://doi.org/10.1109/MCSE.2007.55>
- [81] Edwin L. Hutchins, James D. Hollan, and Donald A. Norman. 1985. Direct Manipulation Interfaces. (1985).
- [82] Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2022. Jigsaw: Large Language Models Meet Program Synthesis. In *Proceedings of the 44th International Conference on Software Engineering*. <https://doi.org/10.1145/3510003.3510203>
- [83] Kelsey Jordahl, Joris Van Den Bossche, Martin Fleischmann, James McBride, Jacob Wasserman, Matt Richards, Adrian Garcia Badaracco, Alan D. Snow, Jeffrey Gerard, Jeff Tratner, Matthew Perry, Brendan Ward, Carson Farmer, Geir Arne Hjelle, Mike Taves, Ewout Ter Hoeven, Micah Cochran, Raymondgh, Sean Gillies, Giacomo Caria, Lucas Culbertson, Matt Bartos, Nick Eubank, Ray Bell, Sangarshanan, John Flavin, Sergio Rey, Maxalbert, Aleksey Bilogur, and Christopher Ren. 2022. Geopandas/Geopandas: V0.12.2. Zenodo. <https://doi.org/10.5281/ZENODO.7422493>
- [84] Kaggle. 2022. State of Machine Learning and Data Science 2022. <https://www.kaggle.com/kaggle-survey-2022>
- [85] Guolin Ke, Qi Meng, Thomas Finely, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems 30 (NIP 2017)*. <https://www.microsoft.com/en-us/research/publication/lightgbm-a-highly-efficient-gradient-boosting-decision-tree/>
- [86] Daniel Keim, Gennady Andrienko, Jean-Daniel Fekete, Carsten Görg, Jörn Kohlhammer, and Guy Melançon. 2008. *Visual Analytics: Definition, Process, and Challenges*.
- [87] KeplerGL. 2019. KeplerGL: A Powerful Open Source Geospatial Analysis Tool for Large-Scale Data Sets. <https://github.com/keplergl/keplergl>
- [88] Andreas Kerren, Kostiantyn Kucher, Yuan-Fang Li, and Falk Schreiber. 2017. BioVis Explorer: A Visual Guide for Biological Data Visualization Techniques. *PLOS ONE* 12 (2017). <https://doi.org/10.1371/journal.pone.0187341>
- [89] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E. John, and Brad A. Myers. 2018. The Story in the Notebook: Exploratory Data Science Using a Literate Programming Tool. In *CHI*. <https://doi.org/10.1145/3173574.3173748>
- [90] Mary Beth Kery, Donghao Ren, Fred Hohman, Dominik Moritz, Kanit Wongsuphasawat, and Kayur Patel. 2020. Mage: Fluid Moves Between Code and Graphical Work in Computational Notebooks. In *CHI*. <https://doi.org/10.1145/3379337.3415842>
- [91] Dominik Kerzel, Birgitta König-Ries, and Samuel Sheeba. 2023. MLProvLab: Provenance Management for Data Science Notebooks. (2023). <https://doi.org/10.18420/BTW2023-66>

- [92] Zak King. 2016. Escher: Build, Share, and Embed Visualizations of Metabolic Pathways. <https://github.com/zakandrewking/escher>
- [93] Rodney Kinney, Chloe Anastasiades, Russell Authur, Iz Beltagy, Jonathan Bragg, Alexandra Buraczynski, Isabel Cachola, Stefan Candra, Yoganand Chandrasekhar, Arman Cohan, Miles Crawford, Doug Downey, Jason Dunkelberger, Oren Etzioni, Rob Evans, Sergey Feldman, Joseph Gorney, David Graham, Fangzhou Hu, Regan Huff, Daniel King, Sebastian Kohlmeier, Bailey Kuehl, Michael Langan, Daniel Lin, Haokun Liu, Kyle Lo, Jaron Lochner, Kelsey MacMillan, Tyler Murray, Chris Newell, Smrita Rao, Shaurya Rohatgi, Paul Sayre, Zejiang Shen, Amanpreet Singh, Luca Soldaini, Shivashankar Subramanian, Amber Tanaka, Alex D. Wade, Linda Wagner, Lucy Lu Wang, Chris Wilhelm, Caroline Wu, Jiangjiang Yang, Angele Zamarron, Madeleine Van Zuylen, and Daniel S. Weld. 2023. The Semantic Scholar Open Data Platform. *arXiv 2301.10140* (2023). <http://arxiv.org/abs/2301.10140>
- [94] Alek Kissinger and John van de Wetering. 2020. PyZX: Large Scale Automated Diagrammatic Reasoning. In *Proceedings 16th International Conference on Quantum Physics and Logic, Chapman University, Orange, CA, USA, 10-14 June 2019 (Electronic Proceedings in Theoretical Computer Science, Vol. 318)*. <https://doi.org/10.4204/EPTCS.318.14>
- [95] Almar Klein. 2016. Flexx: Write Desktop and Web Apps in Pure Python. <https://github.com/flexxui/flexx>
- [96] Thomas Kluyver and others. 2016. Jupyter Notebooks - a Publishing Format for Reproducible Computational Workflows. *ELPUB* (2016). <https://doi.org/10.3233/978-1-61499-649-1-87>
- [97] Mikhail Korobov. 2016. ELIS: A Library for Debugging/Inspecting Machine Learning Classifiers and Explaining Their Predictions. *eli5-org*. <https://github.com/eli5-org/eli5>
- [98] Tobias Krael. 2019. Bamboolib: GUI for Pandas DataFrames. <https://github.com/tkrabel/bamboolib>
- [99] Claire Krause, Bex Dunn, Robbi Bishop-Taylor, Caitlin Adams, Chad Burton, Matthew Alger, Sean Chua, Claire Phillips, Vanessa Newey, Kirill Kouzoubov, Alex Leith, Damien Ayers, Andrew Hicks, and DEA Notebooks contributors. 2021. Digital Earth Australia Notebooks and Tools Repository. <https://doi.org/10.26186/145234>
- [100] Kostiantyn Kucher and Andreas Kerren. 2015. Text Visualization Techniques: Taxonomy, Visual Survey, and Community Insights. In *PacificVis*. <https://doi.org/10.1109/PACIFICVIS.2015.7156366>
- [101] Alexander Kukushkin. 2018. Ipyannotate: Jupyter Widget for Data Annotation. <https://github.com/ipyannotate/ipyannotate>
- [102] Nahyun Kwon, Hannah Kim, Sajjadur Rahman, Dan Zhang, and Estevam Hruschka. 2023. Weedle: Composable Dashboard for Data-Centric NLP in Computational Notebooks. In *Companion Proceedings of the ACM Web Conference 2023*. <https://doi.org/10.1145/3543873.3587330>
- [103] Jupyter Physical Science Lab. 2020. JupyterPiDAQ: Interactive Analog Data Acquisition and Analysis within Jupyter Notebooks Using GUI Tools. Jupyter Physical Science Lab. <https://github.com/JupyterPhysSciLab/JupyterPiDAQ>
- [104] Sandia National Laboratories. 2022. Toyplot: Interactive Plotting for Python. Sandia National Laboratories. <https://github.com/sandialabs/toyplot>
- [105] Marcos Lage, Jorge Piazentin Ono, Daniel Cervone, Justin Chiang, Carlos Dietrich, and Claudio T. Silva. 2016. StatCast Dashboard: Exploration of Spatiotemporal Baseball Data. *IEEE Computer Graphics and Applications* 36 (2016). <https://doi.org/10.1109/MCG.2016.101>
- [106] Michelle S. Lam, Zixian Ma, Anne Li, Izequiel Freitas, Dakuo Wang, James A. Landay, and Michael S. Bernstein. 2023. Model Sketching: Centering Concepts in Early-Stage Machine Learning Model Design. *arXiv 2303.02884* (2023). <https://doi.org/10.1145/3544548.3581290>
- [107] Sam Lau, Ian Drosos, Julia M. Markel, and Philip J. Guo. 2020. The Design Space of Computational Notebooks: An Analysis of 60 Systems in Academia and Industry. In *2020 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. <https://doi.org/10.1109/VL/HCC50065.2020.9127201>
- [108] Samuel Lau and Joshua Hug. 2018. *Nbinteract: Generate Interactive Web Pages from Jupyter Notebooks*. Master's thesis. University of California at Berkeley. <https://www.nbinteract.com/#>
- [109] Doris Jung-Lin Lee, Dixon Tang, Kunal Agarwal, Thyne Boonmark, Caitlyn Chen, Jake Kang, Ujjaini Mukhopadhyay, Jerry Song, Micah Yong, Marti A. Hearst, and Aditya G. Parameswaran. 2021. Lux: Always-on Visualization Recommendations for Exploratory Dataframe Workflows. *Vldb Endowment* 15 (2021). <https://doi.org/10.14778/3494124.3494151>
- [110] Haotian Li, Lu Ying, Haidong Zhang, Yingcai Wu, Huamin Qu, and Yun Wang. 2023. Notable: On-the-fly Assistant for Data Storytelling in Computational Notebooks. In *CHI*. <https://doi.org/10.1145/3544548.3580965>
- [111] Siwei Li, Zhiyan Zhou, Anish Upadhyay, Omar Shaikh, Scott Freitas, Haekyu Park, Zijie J. Wang, Susanta Routay, Matthew Hull, and Duen Horng Chau. 2020. Argo Lite: Open-Source Interactive Graph Exploration and Visualization in Browsers. In *CIKM*. <https://doi.org/10.1145/3340531.3412877>
- [112] Xingjun Li, Yizhi Zhang, Justin Leung, Chengnian Sun, and Jian Zhao. 2023. EDAssistant: Supporting Exploratory Data Analysis in Computational Notebooks with In Situ Code Search and Recommendation. *ACM TIS 13* (2023). <https://doi.org/10.1145/3545995>
- [113] Lightcurve Collaboration, J. V. d. M. Cardoso, C. Hedges, M. Gully-Santiago, N. Saunders, A. M. Cody, T. Barclay, O. Hall, S. Sagar, E. Turtelboom, J. Zhang, A. Tzanidakis, K. Mighell, J. Coughlin, K. Bell, Z. Berta-Thompson, P. Williams, J. Dotson, and G. Barentsen. 2018. Lightcurve: Kepler and TESS Time Series Analysis in Python. *Astrophysics Source Code Library*. <http://adsabs.harvard.edu/abs/2018ascl.soft12013L>
- [114] Yanna Lin, Haotian Li, Leni Yang, Aoyu Wu, and Huamin Qu. 2023. InkSight: Leveraging Sketch Interaction for Documenting Chart Findings in Computational Notebooks. *IEEE Transactions on Visualization and Computer Graphics* (2023). <https://doi.org/10.1109/TVCG.2023.3327170>
- [115] Zhicheng Liu and J T Stasko. 2010. Mental Models, Visual Reasoning and Interaction in Information Visualization: A Top-down Perspective. *IEEE Transactions on Visualization and Computer Graphics* 16 (2010). <https://doi.org/10.1109/TVCG.2010.177>
- [116] Logan. 2023. Nbtutor: Visualize Python Code Execution (Line-by-Line) in Jupyter Notebook Cells. <https://github.com/lpage/nbtutor>
- [117] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. <https://doi.org/10.48550/arXiv.1705.07874>
- [118] Gabi Maeztu. 2016. Neo4jupyter: A Quick Visualization Tool for Jupyter and Neo4j. <https://github.com/mercurio/neo4jupyter>
- [119] Juan Manuel Mauricio. 2017. Pydgrid: Python Distribution Grid Simulator. <https://github.com/pydgrid/pydgrid>
- [120] Matt McCormick, Brianna Major, Laryssa Abdala, Paul Elliott, and Stephen R. Aylward. 2022. InsightSoftwareConsortium/Itkwidgets: Itkwidgets 0.32.5. Zenodo. <https://doi.org/10.5281/ZENODO.7489693>
- [121] Andrew M McNutt, Chenglong Wang, Robert A Deline, and Steven M. Drucker. 2023. On the Design of AI-powered Code Assistants for Notebooks. In *CHI*. <https://doi.org/10.1145/3544548.3580940>
- [122] Sharan B Merriam et al. 2002. Introduction to Qualitative Research. *Qualitative research in practice: Examples for discussion and analysis* 1 (2002).
- [123] Microsoft. 2019. Interpret Community SDK. <https://github.com/interpretml/interpret-community>
- [124] Microsoft. 2020. Responsible AI Toolbox. Microsoft. <https://github.com/microsoft/responsible-ai-toolbox>
- [125] Intuitive Text Mining. 2019. D3fdgraph: D3 Interactive Animated Force-Directed Graphs in a Jupyter Notebook. <https://github.com/intuitivetextmining/d3fdgraph>
- [126] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*. <https://doi.org/10.1145/3287560.3287596>
- [127] Anthony Moi and Nicolas Patry. 2023. HuggingFace's Tokenizers. <https://github.com/huggingface/tokenizers>
- [128] David Munecchika, Zijie J. Wang, Jack Reidy, Josh Rubin, Krishna Gade, Krishnamurthy Kenthapadi, and Duen Horng Chau. 2022. Visual Auditor: Interactive Visualization for Detection and Summarization of Model Biases. In *VIS*. <https://doi.org/10.1109/VIS54862.2022.00018>
- [129] Arpit Narechania, Arjun Srinivasan, and John Stasko. 2021. NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization from Natural Language Queries. *IEEE Transactions on Visualization and Computer Graphics* 27 (2021). <https://doi.org/10.1109/TVCG.2020.3030378>
- [130] Nengo. 2019. Nengo: A Python Library for Creating and Simulating Large-Scale Brain Models. Nengo. <https://github.com/nengo/nengo>
- [131] Hai Nguyen, David A Case, and Alexander S Rose. 2018. NGLview-Interactive Molecular Graphics for Jupyter Notebooks. *Bioinformatics* 34 (2018). <https://doi.org/10.1093/bioinformatics/btx789>
- [132] Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. 2019. InterpretML: A Unified Framework for Machine Learning Interpretability. *arXiv* (2019). <http://arxiv.org/abs/1909.09223>
- [133] NVIDIA. 2021. NVDashboard: A JupyterLab Extension for Displaying Dashboards of GPU Usage. *RAPIDS*. <https://github.com/rapidsai/jupyterlab-nvdashboard>
- [134] Observable. 2021. Observable: Data Visualization Platform. <https://observablehq.com/>
- [135] Jorge Piazentin Ono, Sonia Castelo, Roque Lopez, Enrico Bertini, Juliana Freire, and Claudio Silva. 2021. PipelineProfiler: A Visual Analytics Tool for the Exploration of AutoML Pipelines. *TVCG* 27 (2021). <https://doi.org/10.1109/TVCG.2020.3030361>
- [136] Intelligent Systems Lab Org. 2019. Open3D: Open3D: A Modern Library for 3D Data Processing. <https://github.com/isl-org/Open3D>
- [137] João Palmeiro, Beatriz Malveiro, Rita Costa, David Polido, Ricardo Moreira, and Pedro Bizarro. 2022. Data+Shift: Supporting Visual Investigation of Data Distribution Shifts by Data Scientists. (2022). <https://doi.org/10.2312/EVS.20221097>

- [138] Chris Parmer. 2020. Dash: Data Apps & Dashboards for Python. Plotly. <https://github.com/plotly/dash>
- [139] Jinglin Peng, Weiyuan Wu, Brandon Lockhart, Song Bian, Jing Nathan Yan, Linghao Xu, Zhixuan Chi, Jeffrey M. Rzeszutarski, and Jiannan Wang. 2021. DataPrep.EDA: Task-Centric Exploratory Data Analysis for Statistical Modeling in Python. In *Proceedings of the 2021 International Conference on Management of Data*. <https://doi.org/10.1145/3448016.3457330>
- [140] Giancarlo Perrone, Jose Unpingco, and Haw-minn Lu. 2020. Network Visualizations with Pyvis and VisJS. *arXiv 2006.04951* (2020). <http://arxiv.org/abs/2006.04951>
- [141] Johann Petrak. 2020. Python-Gatenlp: Python Text Processing, Pattern Matching, and NLP Framework. GateNLP. <https://github.com/GateNLP/python-gatenlp>
- [142] Jorge Piazentin Ono, Juliana Freire, and Claudio T. Silva. 2021. Interactive Data Visualization in Jupyter Notebooks. *Comput Sci Eng* 23 (2021). <https://doi.org/10.1109/MCSE.2021.3052619>
- [143] Nicolas Pielawski, Axel Andersson, Christophe Avenel, Andrea Behanova, Eduard Chelebian, Anna Klemm, Fredrik Nysjö, Leslie Solorzano, and Carolina Wahlby. 2022. *TissUMaps 3: Improvements in Interactive Visualization, Exploration, and Quality Assessment of Large-Scale Spatial Omics Data*. Preprint. Bioinformatics. <https://doi.org/10.1101/2022.01.28.478131>
- [144] PixieDust. 2016. PixieDust: Python Helper Library for Jupyter Notebooks. Pixiedust development. <https://github.com/pixiedust/pixiedust>
- [145] Poliaastro. 2019. Czml3: Python 3 Library to Write CZML. <https://github.com/poliaastro/czml3>
- [146] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2019. CatBoost: Unbiased Boosting with Categorical Features. *arXiv* (2019). <http://arxiv.org/abs/1706.09516>
- [147] PyPathway. 2022. PyPathway: A Python Package for Pathway Visualization. <https://github.com/iseekwonderful/PyPathway>
- [148] QuantStack. 2017. Ipysheet: Jupyter Handsontable Integration. QuantStack. <https://github.com/QuantStack/ipysheet>
- [149] QuantStack. 2022. Ipytree: A Tree Widget Using Jupyter-widgets Protocol and jsTree. QuantStack. <https://github.com/QuantStack/ipytree>
- [150] QuSTaR. 2019. Kaleidoscope: Visualizations for Quantum Computing. <https://github.com/QuSTaR/kaleidoscope>
- [151] Bogdana Rakova, Jingying Yang, Henriette Cramer, and Rumman Chowdhury. 2021. Where Responsible AI Meets Reality: Practitioner Perspectives on Enablers for Shifting Organizational Practices. *Proceedings of the ACM on Human-Computer Interaction* 5 (2021). <https://doi.org/10.1145/3449081>
- [152] Henry W. Robbins, Samuel C. Gutekunst, David B. Shmoys, and David P. Williamson. 2023. GILP: An Interactive Tool for Visualizing the Simplex Algorithm. In *SIGCSE*. <https://doi.org/10.1145/3545945.3569815>
- [153] Jim Robinson. 2022. Module for Embedding Igv.js in an IPython Notebook. <https://github.com/igvteam/igv-notebook>
- [154] Adam Rose. 2020. PandasGUI: A GUI for Pandas DataFrames. <https://github.com/adamrose/PandasGUI>
- [155] Sara Brin Rosenthal, Julia Len, Mikayla Webster, Aaron Gary, Amanda Birmingham, and Kathleen M Fisch. 2018. Interactive Network Visualization in Jupyter Notebooks: visJS2jupyter. *Bioinformatics* 34 (2018). <https://doi.org/10.1093/bioinformatics/btx581>
- [156] Philipp Rudiger. 2016. Geoviews: Simple, Concise Geographical Visualization in Python. HoloViz. <https://github.com/holoviz/geoviews>
- [157] Philipp Rudiger. 2021. Panel: A High-Level App and Dashboarding Solution for Python. HoloViz. <https://github.com/holoviz/panel>
- [158] Adam Rule, Aurélien Tabard, and James D. Hollan. 2018. Exploration and Explanation in Computational Notebooks. In *CHI*. <https://doi.org/10.1145/3173574.3173606>
- [159] Pedro Saleiro, Benedict Kuester, Loren Hinkson, Jesse London, Abby Stevens, Ari Anisfeld, Kit T. Rodolfa, and Rayid Ghani. 2019. Aequitas: A Bias and Fairness Audit Toolkit. *arXiv 1811.05577* (2019). <http://arxiv.org/abs/1811.05577>
- [160] Matheus Xavier Sampaio. 2018. PyMove: Python Library to Simplify Queries and Visualization of Trajectories and Other Spatial-Temporal Data. Insight Data Science Lab. <https://github.com/InsightLab/PyMove>
- [161] Alper Sarikaya, Michael Correll, Lyn Bartram, Melanie Tory, and Danyel Fisher. 2019. What Do We Talk About When We Talk About Dashboards? *IEEE Transactions on Visualization and Computer Graphics* 25 (2019). <https://doi.org/10.1109/TVCG.2018.2864903>
- [162] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2017. Vega-Lite: A Grammar of Interactive Graphics. *IEEE Transactions on Visualization & Computer Graphics (Proc. InfoVis)* (2017). <https://doi.org/10.1109/tvcg.2016.2599030>
- [163] Luigi Sbailò, Ádám Fekete, Luca M. Ghiringhelli, and Matthias Scheffler. 2022. The NOMAD Artificial-Intelligence Toolkit: Turning Materials-Science Data into Knowledge and Understanding. *Computational Materials* (2022). <https://doi.org/10.1038/s41524-022-00935-z>
- [164] Daniel Schiff, Bogdana Rakova, Aladdin Ayeshe, Anat Fanti, and Michael Lennon. 2020. Principles to Practices for Responsible AI: Closing the Gap. *arXiv 2006.04707* (2020). <http://arxiv.org/abs/2006.04707>
- [165] Connor Scully-Allison, Ian Lumsden, Katy Williams, Jesse Bartels, Michela Tauber, Stephanie Brink, Abhinav Bhatle, Olga Pearce, and Katherine E. Isaacs. 2022. Designing an Interactive, Notebook-Embedded, Tree Visualization to Support Exploratory Performance Analysis. *arXiv 2205.04557* (2022). <http://arxiv.org/abs/2205.04557>
- [166] Michael Sedlmair, Miriah Meyer, and Tamara Munzner. 2012. Design Study Methodology: Reflections from the Trenches and the Stacks. *IEEE Transactions on Visualization and Computer Graphics* 18 (2012). <https://doi.org/10.1109/TVCG.2012.213>
- [167] Tim Shawver. 2017. Qgrid: An Interactive Grid for Sorting, Filtering, and Editing DataFrames in Jupyter Notebooks. <https://github.com/quantopian/qgrid>
- [168] Carson Sievert, Chris Parmer, Toby Hocking, Scott Chamberlain, Karthik Ram, Marianne Corvellec, and Pedro Despouy. 2017. Plotly: Create Interactive Web Graphics via 'Plotly.js'. 4 (2017). <https://github.com/plotly/plotly.py>
- [169] Carson Sievert and Kenneth Shirley. 2014. LDavis: A Method for Visualizing and Interpreting Topics. In *Proceedings of the Workshop on Interactive Language Learning, Visualization, and Interfaces*. <https://doi.org/10.3115/v1/W14-3110>
- [170] David Simonne, Jérôme Carnis, Clément Atlan, Corentin Chatelier, Vincent Favre-Nicolin, Maxime Dupraz, Steven J. Leake, Edoardo Zatterin, Andrea Resta, Alessandro Coati, and Marie-Ingrid Richard. 2022. *Gwaihir : Jupyter Notebook Graphical User Interface for Bragg Coherent Diffraction Imaging*. *Journal of Applied Crystallography* 55 (2022). <https://doi.org/10.1107/S1600576722005854>
- [171] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. TKET: A Retargetable Compiler for NISQ Devices. *Quantum Science and Technology* 6 (2020). <https://doi.org/10.1088/2058-9565/ab8e92>
- [172] Venkatesh Sivaraman, Yiwei Wu, and Adam Perer. 2022. Emblaze: Illuminating Machine Learning Representations through Interactive Comparison of Embedding Spaces. In *ACM IUI*. <https://doi.org/10.1145/3490099.3511137>
- [173] David H. Smith, Qiang Hao, Christopher D. Hundhausen, Filip Jagodzinski, Josh Myers-Dean, and Kira Jaeger. 2021. Towards Modeling Student Engagement with Interactive Computing Textbooks: An Empirical Study. In *SIGCSE*. <https://doi.org/10.1145/3408877.3432361>
- [174] Jan-Tobias Sohns, Michaela Schmitt, Fabian Jirasek, Hans Hasse, and Heike Leitte. 2022. Attribute-Based Explanation of Non-Linear Embeddings of High-Dimensional Data. *IEEE Transactions on Visualization and Computer Graphics* 28 (2022). <https://doi.org/10.1109/TVCG.2021.3114870>
- [175] Andrew Stein. 2022. Perspective: Interactive Analytics and Data Visualization Component. <https://github.com/finos/perspective>
- [176] R Studio. 2016. R Markdown. <https://rmarkdown.rstudio.com/>
- [177] Ian Tenney, James Wexler, Jasmijn Bastings, Tolga Bolukbasi, Andy Coenen, Sebastian Gehrmann, Ellen Jiang, Mahima Pushkarna, Carey Radebaugh, Emily Reif, and Ann Yuan. 2020. The Language Interpretability Tool: Extensible, Interactive Visualizations and Analysis for NLP Models. In *EMNLP Demo*. <https://doi.org/10.18653/v1/2020.emnlp-demos.15>
- [178] Andreas Tritsarolis, Christos Doukeridis, Nikos Pelekis, and Yannis Theodoridis. 2021. ST_VISIONS: A Python Library for Interactive Visualization of Spatial-temporal Data. In *2021 22nd IEEE International Conference on Mobile Data Management (MDM)*. <https://doi.org/10.1109/MDM52706.2021.00048>
- [179] Uber. 2016. Deck.Gl: WebGL2 Powered Geospatial Visualization Layers. <https://deck.gl>
- [180] C. Upson, T.A. Faulhaber, D. Kamins, D. Laidlaw, D. Schlegel, J. Vroom, R. Gurwitz, and A. Van Dam. 1989. The Application Visualization System: A Computational Environment for Scientific Visualization. *IEEE Computer Graphics and Applications* 9 (1989). <https://doi.org/10.1109/38.31462>
- [181] Jonas Van Der Donckt, Jeroen Van der Donckt, Emiel Deprost, and Sofie Van Hoecke. 2022. Plotly-Resampler: Effective Visual Analytics for Large Time Series. In *2022 IEEE Visualization and Visual Analytics (VIS)*. <https://doi.org/10.1109/VIS54862.2022.00013>
- [182] Jacob VanderPlas, Brian Granger, Jeffrey Heer, Dominik Moritz, Kanit Wongsuphasawat, Arvind Satyanarayan, Eitan Lees, Ilia Timofeev, Ben Welsh, and Scott Sievert. 2018. Altair: Interactive Statistical Visualizations for Python. *Journal of Open Source Software* 3 (2018). <https://doi.org/10.21105/joss.01057>
- [183] Sidheswar Venkatachalapathi. 2020. Quick-EDA: Simple & Easy-to-use Python Modules to Perform Quick Exploratory Data Analysis for Any Structured Dataset. <https://github.com/sid-the-coder/QuickDA>
- [184] Mauricio Verano Merino, Jurgen Vinju, and Tijs van der Storm. 2020. Bacatá: Notebooks for DSLs, Almost for Free. *The Art, Science, and Engineering of Programming* 4 (2020). <https://doi.org/10.22152/programming-journal.org/2020/4/11>
- [185] Jesse Vig. 2019. A Multiscale Visualization of Attention in the Transformer Model. In *ACL: System Demonstrations*. <https://doi.org/10.18653/v1/P19-3007>
- [186] Vizzu. 2022. Ipyvizzu: Build Animated Charts in Jupyter Notebook and Similar Environments with a Simple Python Syntax. Vizzu. <https://github.com/vizzuhq/ipyvizzu>
- [187] Voxel51. 2020. Fiftyone: Building High-Quality Datasets and Computer Vision Models. Voxel51. <https://github.com/voxel51/fiftyone>

- [188] Changhan Wang, Anirudh Jain, Danlu Chen, and Jiatao Gu. 2019. VizSeq: A Visual Analysis Toolkit for Text Generation Tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP): System Demonstrations*. <https://doi.org/10.18653/v1/D19-3043>
- [189] Fengjie Wang, Xuye Liu, Oujing Liu, Ali Neshati, Tengfei Ma, Min Zhu, and Jian Zhao. 2023. Slide4N: Creating Presentation Slides from Computational Notebooks with Human-AI Collaboration. In *CHI*. <https://doi.org/10.1145/3544548.3580753>
- [190] Qiaosi Wang, Michael Madaio, Shaun Kane, Shivani Kapania, Michael Terry, and Lauren Wilcox. 2023. Designing Responsible AI: Adaptations of UX Practice to Meet Responsible AI Challenges. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3544548.3581278>
- [191] Yihan Wang, Yutong Shao, and Ndapa Nakashole. 2021. Interactive Plot Manipulation Using Natural Language. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Demonstrations*. <https://doi.org/10.18653/v1/2021.naacl-demos.11>
- [192] Zijie J. Wang, Katie Dai, and W. Keith Edwards. 2022. StickyLand: Breaking the Linear Presentation of Computational Notebooks. *CHI EA* (2022). <https://doi.org/10.1145/3491101.3519653>
- [193] Zijie J. Wang, Alex Kale, Harsha Nori, Peter Stella, Mark E. Nunnally, Duen Horng Chau, Mihaela Vorvoreanu, Jennifer Wortman Vaughan, and Rich Caruana. 2022. Interpretability, Then What? Editing Machine Learning Models to Reflect Human Knowledge and Values. In *KDD*. <https://doi.org/10.1145/3534678.3539074>
- [194] Zijie J. Wang, Alex Kale, Harsha Nori, Peter Stella, Mark E. Nunnally, Duen Horng Chau, Mihaela Vorvoreanu, Jennifer Wortman Vaughan, and Rich Caruana. 2022. Interpretability, Then What? Editing Machine Learning Models to Reflect Human Knowledge and Values. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*. <https://doi.org/10.1145/3534678.3539074>
- [195] Zijie J. Wang, Chinmay Kulkarni, Lauren Wilcox, Michael Terry, and Michael Madaio. 2024. Farsight: Fostering Responsible AI Awareness During AI Application Prototyping. In *CHI Conference on Human Factors in Computing Systems*. *arXiv* (2022). <http://arxiv.org/abs/2205.03963>
- [196] Zijie J. Wang, David Munechika, Seongmin Lee, and Duen Horng Chau. 2022. NOVA: A Practical Method for Creating Notebook-Ready Visual Analytics. *arXiv* (2022). <http://arxiv.org/abs/2205.03963>
- [197] Zijie J. Wang, Chudi Zhong, Rui Xin, Takuya Takagi, Zhi Chen, Duen Horng Chau, Cynthia Rudin, and Margo Seltzer. 2022. TimberTrek: Exploring and Curating Sparse Decision Trees with Interactive Visualization. In *VIS*. <https://doi.org/10.1109/VIS54862.2022.00021>
- [198] Vincent Warmerdam, Thomas Kober, and Rachael Tatman. 2020. Going beyond T-SNE: Exposing Whatlies in Text Embeddings. In *Proceedings of Second Workshop for NLP Open Source Software (NLP-OSS)*. <https://doi.org/10.18653/v1/2020.nlposs-1.8>
- [199] Weights and Biases. 2021. Weights & Biases: A Tool for Visualizing and Tracking Your Machine Learning Experiments. *Weights & Biases*. <https://github.com/wandb/wandb>
- [200] Nathaniel Weinman, Steven M. Drucker, Titus Barik, and Robert DeLine. 2021. Fork It: Supporting Stateful Alternatives in Computational Notebooks. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3411764.3445527>
- [201] James Wexler, Mahima Pushkarna, Tolga Bolukbasi, Martin Wattenberg, Fernanda Viegas, and Jimbo Wilson. 2019. The What-If Tool: Interactive Probing of Machine Learning Models. *TVCG* 26 (2019). <https://doi.org/10.1109/TVCG.2019.2934619>
- [202] Katy Williams, Alex Bigelow, and Kate Isaacs. 2019. Visualizing a Moving Target: A Design Study on Task Parallel Programs in the Presence of Evolving Data and Concerns. *IEEE Transactions on Visualization and Computer Graphics* (2019). <https://doi.org/10.1109/TVCG.2019.2934285>
- [203] Marc Wouts. 2019. Itables: Pandas DataFrames as Interactive DataTables. <https://github.com/mwouts/itables>
- [204] Aoyu Wu, Dazhen Deng, Furui Cheng, Yingcai Wu, Shixia Liu, and Huamin Qu. 2022. In Defence of Visual Analytics Systems: Replies to Critics. *IEEE Transactions on Visualization and Computer Graphics* (2022). <https://doi.org/10.1109/TVCG.2022.3209360>
- [205] Tongshuang Wu, Marco Tulio Ribeiro, Jeffrey Heer, and Daniel Weld. 2019. Errudite: Scalable, Reproducible, and Testable Error Analysis. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. <https://doi.org/10.18653/v1/P19-1073>
- [206] Yifan Wu, Joseph M. Hellerstein, and Arvind Satyanarayan. 2020. B2: Bridging Code and Interactive Visualization in Computational Notebooks. In *UIST*. <https://doi.org/10.1145/3379337.3415851>
- [207] Peter Xenopoulos, Joao Rulff, Luis Gustavo Nonato, Brian Barr, and Claudio Silva. 2023. Calibrate: Interactive Analysis of Probabilistic Model Output. *TVCG* 29 (2023). <https://doi.org/10.1109/TVCG.2022.3209489>
- [208] Carmen Yip, Jie Mi Chong, Sin Yee Kwek, Yong Wang, and Kotaro Hara. 2021. Visionary Caption: Improving the Accessibility of Presentation Slides Through Highlighting Visualization. In *The 23rd International ACM SIGACCESS Conference on Computers and Accessibility*. <https://doi.org/10.1145/3441852.3476539>
- [209] W. Yu, M. Carrasco Kind, and R.J. Brunner. 2017. Vizic: A Jupyter-based Interactive Visualization Tool for Astronomical Catalogs. *Astronomy and Computing* 20 (2017). <https://doi.org/10.1016/j.ascom.2017.06.004>
- [210] Ashley Zhang, Yan Chen, and Steve Oney. 2023. VizProg: Identifying Misunderstandings By Visualizing Students' Coding Progress. In *CHI*. <https://doi.org/10.1145/3544548.3581516>
- [211] Amy X. Zhang, Michael Muller, and Dakuo Wang. 2020. How Do Data Science Workers Collaborate? Roles, Workflows, and Tools. *Proceedings of the ACM on Human-Computer Interaction* 4 (2020). <https://doi.org/10.1145/3392826>
- [212] Dan Zhang, Hannah Kim, Rafael Li Chen, Eser Kandogan, and Estevam Hruschka. 2023. MEGAnno: Exploratory Labeling for NLP in Computational Notebooks. *arXiv* 2301.03095 (2023). <http://arxiv.org/abs/2301.03095>
- [213] Zhiming Zhao, Spiros Koulouzis, Riccardo Bianchi, Siamak Farshidi, Zeshun Shi, Ruyue Xin, Yuandou Wang, Na Li, Yifang Shi, Joris Timmermans, and W. Daniel Kissling. 2022. Notebook-as-a-VRE (NaaVRE): From Private Notebooks to a Collaborative Cloud Virtual Research Environment. *Software: Practice and Experience* 52 (2022). <https://doi.org/10.1002/spe.3098>

A CHARACTERIZING NOTEBOOK VISUALIZATION TOOLS

Below we characterize 163 collected notebook visualization tools using our organizational framework (targeted users and a four-dimensional design space described in § 3), as well as their supported notebook platforms and implementation methods. See [SUPERNOVA](#) for an interactive version with more details about each entry.

Table 1: This table describes the characterization of 163 notebook visualization tools using our organizational framework. The columns include notebook visualization tools’ names, intended users (data scientists , scientist , educators and students ); visualization-notebook communication styles (no direct communication , one-way , bidirectional ); data source (runtime , text and code , external ); display style (on-demand , always-on ); modularity (monolithic , modular ); their supported notebook platforms (Jupyter Notebook only , JupyterLab only , Jupyter Notebook + JupyterLab , all popular platforms ); and their implementation methods (NOVA , HTML display , ipywidget , Lab Extension , custom servers ).

Notebook Vis Tool	User	Com.	Data	Disp.	Mod.	Plat.	Imp.	Notebook Vis Tool	User	Com.	Data	Disp.	Mod.	Plat.	Imp.
Aequitas [159]								Altair [182]							
Anteater [53]								Apache Beam [7]							
Argo Lite [111]								Atria [202]							
AutoProfiler [49]								AutoViz [11]							
Ax [51]								B2 [206]							
Bacata [184]								Bamboolib [98]							
BERTopic [69]								BertViz [185]							
Bokeh [22]								bqplot [27]							
Brax [66]								Calibrate [207]							
Calling Context Tree [165]								CatBoost [146]							
CausalVis [71]								ChartPy [40]							
Clustergrammar [55]								Cyberhubs [76]							
Cytoscapejs [56]								CZML3 [145]							
d3fdgraph [125]								Data+Shift [137]							
Data-Purifier [72]								datapane [41]							
DataPrep [139]								DEA Tools [99]							
DocML [20]								EDAssistant [112]							
ELI5 [97]								Emblaze [172]							
Errudite [205]								Escher [92]							
Evidently [4]								Farsight [195]							
FiftyOne [187]								Firefly [73]							
Flexx [95]								Folium [54]							
GAM Changer [193]								GateNLP [141]							
GeoPandas [83]								GeoViews [156]							

Table 1: This table describes the characterization of 163 notebook visualization tools using our organizational framework. The columns include notebook visualization tools’ names, intended users (data scientists , scientist , educators and students ); visualization-notebook communication styles (no direct communication , one-way , bidirectional ); data source (runtime , text and code , external ); display style (on-demand , always-on ); modularity (monolithic , modular ); their supported notebook platforms (Jupyter Notebook only , JupyterLab only , Jupyter Notebook + JupyterLab , all popular platforms ); and their implementation methods (NOVA , HTML display , ipywidget , Lab Extension , custom servers ).

Notebook Vis Tool	User	Com.	Data	Disp.	Mod.	Plat.	Imp.	Notebook Vis Tool	User	Com.	Data	Disp.	Mod.	Plat.	Imp.
Lux [109]								mage [90]							
Matplotlib [80]								Mayavi [48]							
MEGAAnno [212]								MLProvLab [91]							
ModelSketchBook [106]								mols2grid [26]							
Moving Pandas [68]								NaaVRE [213]							
nbinteract [108]								Nbtutor [116]							
Nengo [130]								neo4jupyter [118]							
Networkit [5]								NGLview [131]							
Nilearn [3]								NL4DV [129]							
NoLiES [174]								NOMAD [163]							
Notable [110]								NVDashboard [133]							
Open3d [136]								Pandas-Bokeh [77]							
PandasGUI [154]								Panel [157]							
Pathpy [75]								Perspective [175]							
PI2 [36]								Pigeon [62]							
PipelineProfiler [135]								PixieDust [144]							
Plotly [168]								Plotly-Resampler [181]							
Plotting Agent [191]								Py2cytoscape [25]							
py3Dmol [10]								PyCaret [16]							
pydeck [179]								pydgrid [119]							
PyGeoHydro [34]								pyLDavis [169]							
PyMove [160]								PyPathway [147]							
PyPotree [23]								Pytket [171]							
Pyvis [140]								PyZX [94]							
Qgrid [167]															

B DATA COLLECTION DETAILS

To study how researchers and practitioners design interactive visualization tools for computational notebooks, we collected and analyzed 64 academic papers and 105 systems in the wild. We define notebook visualization tools as systems that can display interactive visualizations in Python computational notebooks.

Literature Collection. We searched Google Scholar for notebook visualization tools and performed forward and backward reference searches to snowball the results. The venues of collected papers range from scientific journals (e.g., *Bioinformatics* and *Frontiers in Neuroinformatics*) to human-computer interaction and machine learning conferences (e.g., *VIS*, *CHI*, and *NeurIPS*).

Visualization Package Collection. We first scraped 8.6 million notebooks with `.ipynb` extension from GitHub. Each notebook file is a JSON file containing metadata about the notebook and the notebook cells. The notebook cells contain information about the cell type, the source code or text of the cell, and any output generated by the cell. We pruned the scraped notebooks to only those containing interactive components by searching for script tags in cell outputs of type `text/html`. If a cell was deemed a potential candidate, we extracted the associated source code for that cell. Next, for each candidate interactive notebook, we identified all modules in the notebook by parsing it as an abstract syntax tree and looking for import statements. Finally, we spliced the last line of the source code for the candidate cell into its individual variable components and checked if these matched any of the imported modules or their aliases.

Automating this procedure across 8.6 million notebooks, we built a comprehensive list of 984 Python packages that were potential visualization tools. Since this list of packages contained false positives (not all identified packages were interactive visualization tools), we manually examined each package to verify if it was an interactive visualization tool by looking at the source code and documentation for the package and its usage in notebooks. In total, we identified 105 packages that were also interactive visualization tools.

C IMPLEMENTATION DETAILS

Depending on the need for a backend server, visualization-notebook communication, needed data types, and display styles, there are multiple methods with varying difficulties to implement notebook visualization tools. Note that some methods are only compatible with specific notebook platforms (e.g., JupyterLab, Colab, VSCode, and Kaggle Notebook).

C.1 With Backend Servers

To implement notebook visualization tools that require a backend server, the developer needs to configure the server to support notebooks and establish callback functions to share states with the notebook. The server can either be run directly from the notebook environment or externally. The front-end of the tool can then be displayed in the notebook using the notebook’s native HTML display. It is important to separate the server from the main thread if it is run directly from the notebook to avoid blocking the Python kernel. For example, JUPYTER-DASH [138] and LIT [177] use this method with a Flask backend server and a direct WSGI server, respectively.

C.2 Without Backend Servers

If the tool does not require a server, several implementation methods depend on the visualization-notebook communications.

C.2.1 No Direct Communication. If a web-based visualization tool does not communicate with the notebook environment, the developer can simply use the notebook’s native HTML display to show the tool a notebook cell. The HTML display internally uses `iframe` to embed any web documents.

C.2.2 One-way Communication. To pass data from the notebook Python kernel to the visualization tool, one can use the Web standard’s `postMessage` method to send serialized Python objects as JSON text to the visualization tool’s `iframe`. See NOVA [196] for more details and examples about this approach. Example tools include GAM CHANGER [193] and TIMBERTREK [197].

Alternatively, developers can use existing interactive visualization packages such as PLOTLY [168], BOKEH [22], ALTAIR [182], and PANEL [157] as building blocks to implement their visualization tools. Then, the developer can use these packages’ APIs to pass data from notebooks to the visualization tools. However, this approach is less customizable, and it is best suited for simpler tools. Example tools include INTERPRETML [132] and NILEARN [3].

C.2.3 Bidirectional Communication. To send data back from the visualization tool to the Python kernel, the developer needs to use platform-specific solutions, which vary across platforms because notebook platforms have different security protocols. For Jupyter Notebook and JupyterLab, one can use `ipywidget` with the `comm` protocol to synchronize states between the visualization tool and the notebook. Example tools include MAGE [90] and PYDEC [179].

C.3 Access and Modify Code and Text

To access and modify notebook content outside of the Python kernel, such as raw code and text (§ 5.2), visualization tool developers need to use platform-specific APIs. For Jupyter notebooks, the developer can use Jupyter Notebook extension and JupyterLab extension APIs to read and write the notebook content. visualization tools using this method include B2 [206] and WREX [44].

C.4 Always-on Display

If a developer intends to implement an always-on display (§ 5.3) for their notebook visualization tool, they can use platform-specific APIs. For JupyterLab, the developer can implement the tool as a JupyterLab extension, which enables the display on persistent panels outside of the notebook’s main UI. Examples of such implementations include NVDASHBOARD [133] and AUTOProfiler [49]. If the visualization tool does not require extensive visualization customization, the developer can also use existing visualization packages that support persistent display (e.g., JUPYTER-DASH [138]) to implement the tool. Alternatively, the developer can develop their visualization tool using a traditional on-demand display and instruct users to use STICKYLAND [192] to enable persistent display. STICKYLAND allows users to easily create persistent “sticky” cells and dashboards by dragging any notebook cell to the edge of the notebook’s UI (Fig. 11).